

数据加工基础及数据可视化

在数据分析和数据科学项目中，常用的基础包有：

Numpy：数组处理

Pandas：数据加工

Scikit-learn：机器学习

Matplotlib：统计可视化

Seaborn：数据可视化

StatsModels：统计分析

pandasql:SQL编程

本节中主要介绍Numpy、Pandas数据处理和Matplotlib、Seaborn可视化。

1. 搜索路径

1.1. 变量搜索路径

dir() 或者 %whos

```
In [1]: dir() #先显示jupyter notebook的变量，后显示自己定义的变量
```

```
Out[1]: ['In',
         'Out',
         '_',
         '_',
         '_',
         '_',
         '__builtin__',
         '__builtins__',
         '__doc__',
         '__loader__',
         '__name__',
         '__package__',
         '__spec__',
         '_dh',
         '_i',
         '_il',
         '_ih',
         '_ii',
         '_iii',
         '_oh',
         'exit',
         'get_ipython',
         'quit']
```

```
In [2]: s1 = 110
dir()
```

```
Out[2]: ['In',
         'Out',
         ',',
         '_',
         '_1',
         '_',
         '_',
         '__builtin__',
         '__builtins__',
         '__doc__',
         '__loader__',
         '__name__',
         '__package__',
         '__spec__',
         '_dh',
         '_i',
         '_il',
         '_i2',
         '_ih',
         '_ii',
         '_iii',
         '_oh',
         'exit',
         'get_ipython',
         'quit',
         's1']
```

```
In [3]: # 将某个变量从搜索路径中删除的方法：用del语句
del s1
```

```
In [4]: s1
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-4-d0257f733e5e> in <module>()
----> 1 s1

NameError: name 's1' is not defined
```

1.2 模块搜索路径

模块的搜索过程是先在python解释器中搜索，再到用户定义\下载\安装的模块中搜索

```
In [5]: import sys
sys.path
# 注：sys.path是属性，而不是方法，不能加括号()。
```

```
Out[5]: ['',
         'D:\\ProgramData\\Anaconda3\\python36.zip',
         'D:\\ProgramData\\Anaconda3\\DLLs',
         'D:\\ProgramData\\Anaconda3\\lib',
         'D:\\ProgramData\\Anaconda3',
         'C:\\Users\\Admin\\AppData\\Roaming\\Python\\Python36\\site-packages',
         'D:\\ProgramData\\Anaconda3\\lib\\site-packages',
         'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\wordcloud-1.8.1-py3.6-win-amd64.egg',
         'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\win32',
         'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\win32\\lib',
         'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\Pythonwin',
         'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\IPython\\extensions',
         'C:\\Users\\Admin\\.ipython']
```

```
In [6]: import sys
sys.path.append('H:\\Python\\Anaconda') #增加一个新路径至模块搜索路径
```

```
In [7]: sys.path
```

```
Out[7]: ['D:\\ProgramData\\Anaconda3\\python36.zip',
'D:\\ProgramData\\Anaconda3\\DLLs',
'D:\\ProgramData\\Anaconda3\\lib',
'D:\\ProgramData\\Anaconda3',
'C:\\Users\\Admin\\AppData\\Roaming\\Python\\Python36\\site-packages',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\wordcloud-1.8.1-py3.6-win-amd64.egg',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\win32',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\win32\\lib',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\Pythonwin',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\IPython\\extensions',
'C:\\Users\\Admin\\.ipython',
'H:\\Python\\Anaconda']
```

```
In [8]: sys.path.remove('H:\\Python\\Anaconda') #从模块搜索路径中删除一个路径
```

```
In [9]: sys.path
```

```
Out[9]: ['D:\\ProgramData\\Anaconda3\\python36.zip',
'D:\\ProgramData\\Anaconda3\\DLLs',
'D:\\ProgramData\\Anaconda3\\lib',
'D:\\ProgramData\\Anaconda3',
'C:\\Users\\Admin\\AppData\\Roaming\\Python\\Python36\\site-packages',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\wordcloud-1.8.1-py3.6-win-amd64.egg',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\win32',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\win32\\lib',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\Pythonwin',
'D:\\ProgramData\\Anaconda3\\lib\\site-packages\\IPython\\extensions',
'C:\\Users\\Admin\\.ipython']
```

2.当前工作目录

2.1 显示当前工作目录的方法

```
In [10]: import os
print(os.getcwd()) #显示当前工作目录: C:\\Users\\Admin\\深度学习基础（一）2021-2022\\第一次上机内容
```

C:\\Users\\Admin\\深度学习基础（一）2021-2022\\第一次上机内容

2.2 更改当前工作目录的方法

```
In [11]: os.chdir('D:\\win10_vmware') # 注意要保证目录存在
print(os.getcwd())
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
<ipython-input-11-21201c8daab7> in <module>()
----> 1 os.chdir('D:\\win10_vmware') # 注意要保证目录存在
      2 print(os.getcwd())
```

FileNotFoundError: [WinError 2] 系统找不到指定的文件。: 'D:\\win10_vmware'

```
In [12]: os.chdir(r'C:\\Users\\Admin\\深度学习基础（一）2021-2022\\第一次上机内容') #将目录换回到原文件和数据存储的目录: C:\\U
print(os.getcwd())
```

C:\\Users\\Admin\\深度学习基础（一）2021-2022\\第一次上机内容

2.3 读写当前工作目录的方法

```
In [ ]: from pandas import read_csv
data = read_csv('bc_data.csv')
data.head(5)
#前提是将目标文件“bc_data.csv”已经放到当前工作目录中，如：C:\Users\pro7\人文-计算传播基础\数据转换和基本清洗
```

3. 随机数

3.1 一次生成一个数

```
In [13]: import random
random.seed(3) # random.seed()的功能为生成随机数的“种子数”。
```

```
In [14]: import random
random.seed(3) # 如果不设置参数，系统则每次生成的随机数都不一样。
random.randint(1,100) #生成一个[1,100]之间的随机整数。
```

Out[14]: 31

```
In [15]: import random
random.seed(3)
random.uniform(-10,10) #生成一个[-10,10]之间的随机浮点数（实数）。
```

Out[15]: -5.240707458162173

```
In [16]: random.uniform?
```

```
In [17]: random.seed(3)
round(random.uniform(-10,10),2) #其中“2”的含义为小数点后面保留两个有效位
```

Out[17]: -5.24

```
In [18]: import random
random.seed(3)
random.uniform(5,6)
```

Out[18]: 5.237964627091891

3.2 一次生成一个随机数组

用NumPy包，生成一个随机数组，步骤如下：

- 第一步，通过np.random.RandomState定义一个随机变量的生成器rand
- 第二步，根据目标数组的特征（如服从均匀分布还是正态分布），选择生成器rand提供的具体方法，如rand.randint()、rand.rand()、rand.randn()

(1) 生成整数，如生成一个 3×6 的矩阵，矩阵的每个元素的取值范围为[0,10]之间的整数。

```
In [19]: import numpy as np
rand = np.random.RandomState(32)
x = rand.randint(0,10,(3,6))#此处，np.random.RandomState()的参数32为随机数种子数，(3,6)为目标数组的形状。
x
```

Out[19]: array([[7, 5, 6, 8, 3, 7],
[9, 3, 5, 9, 4, 1],
[3, 1, 2, 3, 8, 2]])

(2) 生成服从均匀分布的浮点数（实数）数组，如生成一个含有5个元素的数组。

```
In [20]: import numpy as np
rand = np.random.RandomState(1)
x = rand.rand(5)*10
# rand.rand()的返回值的取值范围为[0,1],在此“*10”的目的是“调整所生成随机数的取值范围”
x
```

Out[20]: array([4.17022005e+00, 7.20324493e+00, 1.14374817e-03, 3.02332573e+00,
1.46755891e+00])

(3) 生成服从正态分布的浮点数（实数）数组，如生成一个含有5个元素且服从正态分布的数组。

```
In [21]: import numpy as np
        rand = np.random.RandomState(1)
        y = rand.randn(5)+5
        y
```

```
Out[21]: array([6.62434536, 4.38824359, 4.47182825, 3.92703138, 5.86540763])
```

(4) 生成等距数列，如产生一个含有20个元素的等距数列，其中每个元素的取值范围为[0,10]。

```
In [22]: x = np.linspace(0,10,20)
        x
```

```
Out[22]: array([ 0.          ,  0.52631579,  1.05263158,  1.57894737,  2.10526316,
                2.63157895,  3.15789474,  3.68421053,  4.21052632,  4.73684211,
                5.26315789,  5.78947368,  6.31578947,  6.84210526,  7.36842105,
                7.89473684,  8.42105263,  8.94736842,  9.47368421, 10.          ])
```

```
In [23]: # 可以查阅np.linspace的说明文档，了解更多内容
        np.linspace?
```

4. 数组

- 关于Python中的“数组”的背景知识：

(1) Python语言的基础语法中并没有提供数组类型，数组的功能可以用列表和元组来实现。

(2) 用列表或元组来代替C和Java语言中的数组的缺点——时间和空间代价都很大。

原因：在Python中，列表和元组中的每个元素都是按“对象”来处理的，每个成员都需要存储引用和对象值。

(3) Python中出现了“以优化列表和元组，进而实现数组功能”的第三方扩展包，如NumPy等。注：另有一个扩展模块叫array，但一般不用它。

(4) 最常用的是Python扩展包NumPy中的数据结构ndarray。相对于列表和元组，ndarray有三个优点：更节省内存、更节省运行时间、更方便使用。

原因：NumPy是用C实现的，并进行了一定的优化处理。

(5) ndarray的本质是n维数组。特殊性体现在“支持通过参数dtype设置数组元素的类型”。

【备注】

列表和元组对计算机的内存（空间复杂度）和cpu（时间复杂度）消耗较大。

列表和元组的功能可以由numpy来实现，大数据分析时使用numpy来实现。

```
In [24]: # 在调用ndarray之前需要导入NumPy模块。
        import numpy as np
```

4.1 创建方法

ndarray对象的创建方法有多种：

- 第一类方法：用np.arange()
- 第二类方法：用np.array()
- 第三类方法：用np.zeros()、np.ones()等函数
- 第四类方法：用np.full()创建相同元素的数组
- 第五类方法：用np.random()生成随机数组

```
In [25]: MyArray1 = np.arange(1,20)
        MyArray1
```

```
Out[25]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
                18, 19])
```

```
In [26]: range(1, 10, 2)    #很长的数组不建议使用该方法
```

```
Out[26]: range(1, 10, 2)
```

【提示】：从输出结果看，range()的返回值为一个迭代器。然后使用list强制类型转换即可显示range()返回的迭代器中的内容。

```
In [27]: list(range(1, 10, 2))    #很长的数组不建议使用该方法
```

```
Out[27]: [1, 3, 5, 7, 9]
```

```
In [28]: np.arange(1, 10, 2)
```

```
Out[28]: array([1, 3, 5, 7, 9])
```

【注意】：

np.arange(1,20)和range(1,20)的功能类似，返回一个由大小等于1（包含1）且小于20（不包含20）的自然数组成的有序数组。

range()与arange()的区别：

- range是Python内置函数，返回值为range对象
- arange是扩展模块numpy中的函数，返回值为NumPy的Array

虽然两者的功能相同，但是后者是NumPy的方法，运行速度更快，占用内存更小，用起来更方便。

```
In [29]: MyArray2=np.array([1,2,3,4,3,5])    #array()是模块NumPy中的函数。  
MyArray2
```

```
Out[29]: array([1, 2, 3, 4, 3, 5])
```

```
In [30]: np.array(range(1, 10, 2))    #这种写法不建议，应该改为下面的写法  
np.arange(1, 10, 2)
```

```
Out[30]: array([1, 3, 5, 7, 9])
```

```
In [31]: MyArray3 = np.zeros((5,5))  
#zeros外面的括弧是函数zero的括弧，里面的括弧是元组的容器大小，即代表目标数组的形状(shape)，如5行5列的数组。  
MyArray3
```

```
Out[31]: array([[0., 0., 0., 0., 0.],  
               [0., 0., 0., 0., 0.],  
               [0., 0., 0., 0., 0.],  
               [0., 0., 0., 0., 0.],  
               [0., 0., 0., 0., 0.]])
```

```
In [32]: MyArray4 = np.ones((5,5))  
MyArray4
```

```
Out[32]: array([[1., 1., 1., 1., 1.],  
               [1., 1., 1., 1., 1.],  
               [1., 1., 1., 1., 1.],  
               [1., 1., 1., 1., 1.],  
               [1., 1., 1., 1., 1.]])
```

```
In [33]: np.full((3,5),2)
```

```
Out[33]: array([[2, 2, 2, 2, 2],  
               [2, 2, 2, 2, 2],  
               [2, 2, 2, 2, 2]])
```

```
In [34]: rand = np.random.RandomState(30)  
MyArray5 = rand.randint(0,100,[3,5])  
# 此处，0和100代表的是随机数的取值范围，“3,5”代表的是目标数组的形状，即3行5列。  
MyArray5
```

```
Out[34]: array([[37, 37, 45, 45, 12],  
               [23, 2, 53, 17, 46],  
               [ 3, 41, 7, 65, 49]])
```

```
In [35]: rand = np.random.RandomState(30)
MyArray6 = rand.randint(0,100, (3,5))
MyArray6
```

```
Out[35]: array([[37, 37, 45, 45, 12],
                [23,  2, 53, 17, 46],
                [ 3, 41,  7, 65, 49]])
```

4.2 主要特征

ndarray的两个重要特征如下：

- (1) shape: 多维数组的形状

其取值为一个元组或列表，例如，shape = (2,15)代表的是一个2行15列的数组

- (2) dtype:多维数组中的元素的数据类型

其取值为np.int等numpy模块提供的数据类型，例如：dtype = np.int,代表的是数组元素为numpy模块中的int型

```
In [36]: import numpy as np
MyArray4 = np.zeros(shape = (2,15), dtype = np.int)
MyArray4
```

```
Out[36]: array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]])
```

shape参数代表的是数组的形状，取值可以为元组，如(3,5),也可以为列表，如[3,5]。

```
In [37]: np.ones((3,5), dtype = float)
```

```
Out[37]: array([[1., 1., 1., 1., 1.],
                [1., 1., 1., 1., 1.],
                [1., 1., 1., 1., 1.]])
```

```
In [38]: np.ones([3,5], dtype = float)
```

```
Out[38]: array([[1., 1., 1., 1., 1.],
                [1., 1., 1., 1., 1.],
                [1., 1., 1., 1., 1.]])
```

4.3 切片与读取

```
In [39]: import numpy as np
myArray = np.array(range(1,10))
myArray
```

```
Out[39]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [40]: myArray = np.arange(1,10)
myArray
```

```
Out[40]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [41]: myArray[0] # 第一个元素的下标为0
```

```
Out[41]: 1
```

```
In [42]: myArray[-1]
```

```
Out[42]: 9
```

```
In [43]: import numpy as np
myArray=np.array(range(0,10))
# 查看数组myArray的当前值
print("myArray=", myArray)

myArray= [0 1 2 3 4 5 6 7 8 9]
```

```
In [44]: # 1、9、2分别为切片操作的开始位置（start）、结束位置（stop-1）和步长（step）
print("myArray[1:9:2]=" , myArray[1:9:2])
```

```
myArray[1:9:2]= [1 3 5 7]
```

```
In [45]: # 也可以省略start
print("myArray[:9:2]=" , myArray[:9:2])
```

```
myArray[:9:2]= [0 2 4 6 8]
```

```
In [46]: # 可以省略start和stop
print("myArray[:,2]=" , myArray[:,2])
```

```
myArray[:,2]= [0 2 4 6 8]
```

```
In [47]: # start、stop和step都可以省略
print("myArray[:]=" , myArray[:])
```

```
myArray[:]= [0 1 2 3 4 5 6 7 8 9]
```

```
In [48]: # 可以省略 start和step
print("myArray[:8]=" , myArray[:8:])
```

```
myArray[:8]= [0 1 2 3 4 5 6 7]
```

```
In [49]: #可以省略step
print("myArray[:8]=" , myArray[0:8])
```

```
myArray[:8]= [0 1 2 3 4 5 6 7]
```

```
In [50]: #可以省略stop和step
print("myArray[4::]=" , myArray[4::])
```

```
myArray[4::]= [4 5 6 7 8 9]
```

```
In [51]: # step的值可以是负数
print("myArray[9:1:-2]=" , myArray[9:1:-2])
print("myArray[::-2]=" , myArray[::-2])
```

```
myArray[9:1:-2]= [9 7 5 3]
```

```
myArray[::-2]= [9 7 5 3 1]
```

```
In [52]: # 下标中还可能出现另一个[],即[]可以嵌套, 含义为“非连续性读取元素”
print("myArray[[2,5,6]]=" , myArray[[2,5,6]]) # 切片中中括号里的中括号是指要切的下标, 是一种Fancy Indexing
```

```
myArray[[2,5,6]]= [2 5 6]
```

```
In [53]: # 下标中还可能出现含有数组名本身的表达式, 含义为“过滤”
print("myArray[myArray>5]=" , myArray[myArray>5])
```

```
myArray[myArray>5]= [6 7 8 9]
```

```
In [54]: # 切片操作中, 规则为“左包含右不包含”
myArray[0:2]
```

```
Out[54]: array([0, 1])
```

```
In [55]: ## 【注意】: 切片操作时, 数组本身不会发生改变
myArray
```

```
Out[55]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [56]: # 数组的非连续元素的读取方法: 用“切片”
myArray = np.array(range(1,11))
myArray
```

```
Out[56]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
In [57]: #当下标为不规则时，不用Fancy Indexing则会出错。
myArray[1,3,6] # 报错信息：IndexError: too many indices for array (索引的维度过多的错误信息)

-----
IndexError                                Traceback (most recent call last)
<ipython-input-57-02781c3c7fbf> in <module>()
      1 #当下标为不规则时，不用Fancy Indexing则会出错。
----> 2 myArray[1,3,6] # 报错信息：IndexError: too many indices for array (索引的维度过多的错误信息)

IndexError: too many indices for array
```

```
In [58]: # 切片报错，需要 “[1,3,6]” 改为 “[[1,3,6]]”，即采用切片
# 【技巧】：切片的特点是下标中嵌套出现方括号，如 [[]]
myArray[[1,3,6]]
```

```
Out[58]: array([2, 4, 7])
```

```
In [59]: myArray
```

```
Out[59]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

【注】：在数据分析和数据科学项目中，通常需要生成一个特殊矩阵，即“特征矩阵”。

从上面的输出结果看，myArray的当前值为一行记录，不符合“特征矩阵的要求”，需要对其进行规整化处理。

规整化处理特征矩阵的方法如下：

```
In [60]: myArray[:,np.newaxis] # np.newaxis 表示增加一个维度，
                                # 切片[:, ]中的，用来隔开维度
# 【注】：此处的“:”不能省略。
```

```
Out[60]: array([[ 1],
                [ 2],
                [ 3],
                [ 4],
                [ 5],
                [ 6],
                [ 7],
                [ 8],
                [ 9],
                [10]])
```

```
In [61]: # 查看形状用属性shape
myArray[:,np.newaxis].shape
```

```
Out[61]: (10, 1)
```

```
In [62]: # 更改形状用NumPy中的方法reshape()
myArray2 = np.arange(1,21).reshape([5,4])
myArray2
```

```
Out[62]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12],
                [13, 14, 15, 16],
                [17, 18, 19, 20]])
```

```
In [63]: myArray2 = np.arange(1,21).reshape((5,4))
myArray2
```

```
Out[63]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12],
                [13, 14, 15, 16],
                [17, 18, 19, 20]])
```

```
In [64]: # 多维数组的切片读取方法示例如下:
myArray2[[2,4],3] # 切数组的第2行和第4行的第3列
```

```
Out[64]: array([12, 20])
```

```
In [65]: # 另一种更好的多维数组的切片读取方法
x = [2,4]
myArray2[x,3]
```

```
Out[65]: array([12, 20])
```

4.4 浅拷贝与深拷贝

- 浅拷贝——复制过来的是“引用”，即“复制对象和被复制对象共享一个存储空间，二者并不是相互独立的”。
- 深拷贝——复制过来的是“值”，即“复制对象和被复制对象占用两个不同空间，二者是相互独立的”。
- ndarray的深拷贝方法：`.copy()`

```
In [66]: import numpy as np
myArray1 = np.array(range(0,10))
myArray2 = myArray1
print(myArray2)
myArray2[1] = 100 #此处修改了myArray2的取值
print(myArray1)
print(myArray2)
```

```
[0 1 2 3 4 5 6 7 8 9]
[ 0 100  2  3  4  5  6  7  8  9]
[ 0 100  2  3  4  5  6  7  8  9]
```

```
In [67]: import numpy as np
myArray1 = np.arange(0,10)
myArray2 = myArray1.copy()
print(myArray2)
myArray2[1] = 200 #此处修改了myArray2的取值
print(myArray1)
print(myArray2)
```

```
[0 1 2 3 4 5 6 7 8 9]
[0 1 2 3 4 5 6 7 8 9]
[ 0 200  2  3  4  5  6  7  8  9]
```

4.5 形状与重构

(1) 重构 (reshape) 的含义为“返回一个符合新形状要求的数组”

```
In [68]: import numpy as np
MyArray5 = np.arange(1,21)
MyArray5
```

```
Out[68]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
        18, 19, 20])
```

```
In [69]: MyArray5.shape
```

```
Out[69]: (20,)
```

```
In [70]: MyArray6 = MyArray5.reshape(4,5) # reshape() 返回另一个新的数组
MyArray6
```

```
Out[70]: array([[ 1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10],
        [11, 12, 13, 14, 15],
        [16, 17, 18, 19, 20]])
```

```
In [71]: MyArray5.shape
```

```
Out[71]: (20,)
```

```
In [72]: MyArray5
```

```
Out[72]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
               18, 19, 20])
```

```
In [73]: MyArray5.reshape(5,4)  #返回另一个新的5*4的二维数组
```

```
Out[73]: array([[ 1,  2,  3,  4],
               [ 5,  6,  7,  8],
               [ 9, 10, 11, 12],
               [13, 14, 15, 16],
               [17, 18, 19, 20]])
```

```
In [74]: MyArray5
```

```
Out[74]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
               18, 19, 20])
```

```
In [75]: MyArray5.reshape(5,5)  #报错 ValueError: cannot reshape array of size 20 into shape (5,5)
#【原因分析】：reshape的前提是“可以reshape”
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-75-602e33ac2d20> in <module>()
----> 1 MyArray5.reshape(5,5)  #报错 ValueError: cannot reshape array of size 20 into shape (5,5)
      2 #【原因分析】：reshape的前提是“可以reshape”

ValueError: cannot reshape array of size 20 into shape (5,5)
```

```
In [76]: MyArray5
```

```
Out[76]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
               18, 19, 20])
```

(2) `resize()`方法更改数组本身的形状，即“就地修改”

```
In [77]: MyArray5.resize(4,5)  #resize 是就地修改值
MyArray5
```

```
Out[77]: array([[ 1,  2,  3,  4,  5],
               [ 6,  7,  8,  9, 10],
               [11, 12, 13, 14, 15],
               [16, 17, 18, 19, 20]])
```

【注意】：`resize`与`reshape`的区别：前者修改数组本身（即“就地修改”），后者不修改数组本身（即返回另一个新数组）。

(3) `swapaxes()`方法进行轴调换，如实现矩阵转置操作

```
In [78]: MyArray5.swapaxes(0,1)  #swapaxes 表示转置,不改变数组本身，是另建值修改
```

```
Out[78]: array([[ 1,  6, 11, 16],
               [ 2,  7, 12, 17],
               [ 3,  8, 13, 18],
               [ 4,  9, 14, 19],
               [ 5, 10, 15, 20]])
```

```
In [79]: MyArray5
```

```
Out[79]: array([[ 1,  2,  3,  4,  5],
               [ 6,  7,  8,  9, 10],
               [11, 12, 13, 14, 15],
               [16, 17, 18, 19, 20]])
```

```
In [80]: MyArray5 = MyArray5.swapaxes(0,1)
MyArray5
```

```
Out[80]: array([[ 1,  6, 11, 16],
               [ 2,  7, 12, 17],
               [ 3,  8, 13, 18],
               [ 4,  9, 14, 19],
               [ 5, 10, 15, 20]])
```

(4) 用flatten()方法将多维数组转换成一维数组

```
In [81]: MyArray5.flatten()
```

```
Out[81]: array([ 1,  6, 11, 16,  2,  7, 12, 17,  3,  8, 13, 18,  4,  9, 14, 19,  5,
                10, 15, 20])
```

(5) 用tolist()方法将多维数组转换为嵌套列表

```
In [82]: MyArray5.tolist()
```

```
Out[82]: [[1, 6, 11, 16],
           [2, 7, 12, 17],
           [3, 8, 13, 18],
           [4, 9, 14, 19],
           [5, 10, 15, 20]]
```

(6) 可以重设数组元素的数据类型

```
In [83]: MyArray5.astype(np.float)    # .astype 强制转换类型 ,
```

```
Out[83]: array([[ 1.,  6., 11., 16.],
               [ 2.,  7., 12., 17.],
               [ 3.,  8., 13., 18.],
               [ 4.,  9., 14., 19.],
               [ 5., 10., 15., 20.]])
```

```
In [84]: # 【注】： MyArray5.astype(np.float)之后，数组MyArray5本身没有变，而是返回另一个新数组。
MyArray5
```

```
Out[84]: array([[ 1,  6, 11, 16],
               [ 2,  7, 12, 17],
               [ 3,  8, 13, 18],
               [ 4,  9, 14, 19],
               [ 5, 10, 15, 20]])
```

4.6 属性计算

(1) 计算数组的秩——rank()或ndim()

```
In [85]: np.rank(MyArray5)    # rank版本问题，用.ndim代替
```

```
D:\ProgramData\Anaconda3\lib\site-packages\ipykernel_launcher.py:1: VisibleDeprecationWarning: `rank` is deprecated; use the `ndim` attribute or function instead. To find the rank of a matrix see `numpy.linalg.matrix_rank`.
"""Entry point for launching an IPython kernel.
```

```
Out[85]: 2
```

```
In [86]: np.ndim(MyArray5)
```

```
Out[86]: 2
```

```
In [87]: MyArray5.ndim
```

```
Out[87]: 2
```

【提示】：ndim的用法有两种：属性形式，MyArray5.ndim；方法形式，np.ndim(MyArray5)

(2) 数组的形状——.shape()

```
In [88]: np.shape(MyArray5)
```

```
Out[88]: (5, 4)
```

```
In [89]: MyArray5.shape #shape 属性支持函数式调用，如np.shape(MyArray5)等同于MyArray5.shape
```

```
Out[89]: (5, 4)
```

(3) 计算元素个数——.size

```
In [90]: MyArray5.size
```

```
Out[90]: 20
```

(4) 查看数组类型——内置函数type()

```
In [91]: type(MyArray5)
#type 不是NumPy提供的，而是Python内置函数，所以不能加前缀np
```

```
Out[91]: numpy.ndarray
```

4.7 ndarray的计算

(1) 数组的乘法

```
In [92]: MyArray5*10
```

```
Out[92]: array([[ 10,  60, 110, 160],
               [ 20,  70, 120, 170],
               [ 30,  80, 130, 180],
               [ 40,  90, 140, 190],
               [ 50, 100, 150, 200]])
```

(2) 横向拆分——split()方法

```
In [93]: x = np.array([11,12,13,14,15,16,17,18])
x1,x2,x3 = np.split(x, [3,5]) # [3,5]为拆分位置的索引，split是把x在3和5的位置进行切开，切完后元组变成了列表
print(x1, x2, x3)
```

```
[11 12 13] [14 15] [16 17 18]
```

纵向拆分——vsplit()方法，此处为元组的拆包式赋值

```
In [94]: upper, lower = np.vsplit(MyArray5.reshape(4,5), [2])
print("上半部分为\n", upper)
print("\n\n下半部分为\n", lower)
```

```
上半部分为
[[ 1  6 11 16  2]
 [ 7 12 17  3  8]]
```

```
下半部分为
[[13 18  4  9 14]
 [19  5 10 15 20]]
```

数组的合并——np.concatenate()

```
In [95]: np.concatenate((lower, upper), axis = 0)
```

```
Out[95]: array([[13, 18,  4,  9, 14],
               [19,  5, 10, 15, 20],
               [ 1,  6, 11, 16,  2],
               [ 7, 12, 17,  3,  8]])
```

axis = 0 表示计算之后列的个数不变，以列为单位进行计算，逐列合并，即上下合并；

axis = 1 表示计算之后行的个数不变，以行为单位进行计算，逐行合并，即左右合并；

(4) `np.vstack()`和`np.hstack()`分别支持横向或纵向合并

```
In [96]: np.vstack([upper, lower])    # 等同于上下合并，即按列合并；列的个数一样
```

```
Out[96]: array([[ 1,  6, 11, 16,  2],
               [ 7, 12, 17,  3,  8],
               [13, 18,  4,  9, 14],
               [19,  5, 10, 15, 20]])
```

```
In [97]: np.hstack([upper, lower])    # 等同于左右合并，即按行合并；行的个数一样
```

```
Out[97]: array([[ 1,  6, 11, 16,  2, 13, 18,  4,  9, 14],
               [ 7, 12, 17,  3,  8, 19,  5, 10, 15, 20]])
```

```
In [98]: np.add(MyArray5, 1)
```

```
Out[98]: array([[ 2,  7, 12, 17],
               [ 3,  8, 13, 18],
               [ 4,  9, 14, 19],
               [ 5, 10, 15, 20],
               [ 6, 11, 16, 21]])
```

【注意】：同一个功能，用`sum`和`np.sum()`都可以实现，但是：

- 前者为Python内置函数，后者为numpy包的函数，属于ufunc类函数
- ufunc的速度比普通函数快
- `sum`为内置函数，并非为ufunc
- `np.sum`为ufunc函数

【提示】：在NumPy中，数组的函数计算往往为“ufunc类函数”——以列为单位计算的函数，目的是支持向量计算，不用写循环语句。

4.8 ndarray的元素类型

【注意】：

- ndarray中的数组支持用户自定义元素的类型，定义方法——设置`dtype`参数值
- 统一数组中，所有元素的数据类型必须一致

如果不一致或没有显式定义元素数据类型，则按`dtype = object`来处理

```
In [99]: import numpy as np
         np.zeros(10, dtype = "int16")
```

```
Out[99]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int16)
```

```
In [100]: np.zeros(10, dtype = "float")
```

```
Out[100]: array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
In [101]: a1 = np.array([1, 2, 3, None])    #python中列表里可以存放不同类型的数据，None不能参与计算，如None+1会报错。
         a1
```

```
Out[101]: array([1, 2, 3, None], dtype=object)
```

```
In [102]: a1 = np.array([1, 2, 3, None, np.nan])    # np.nan 是jupyternotebook中的写法，在numpy中np.nan可以参与计算。
         a1
```

```
Out[102]: array([1, 2, 3, None, nan], dtype=object)
```

4.9 插入与删除

- 删除特定元素的方法——`np.delete()`
- 插入特定元素的方法——`np.insert()`

```
In [103]: import numpy as np
myArray1 = np.array([11, 12, 13, 14, 15, 16, 17, 18])
np.delete(myArray1, 2)
```

```
Out[103]: array([11, 12, 14, 15, 16, 17, 18])
```

```
In [104]: np.insert(myArray1, 1, 88)
```

```
Out[104]: array([11, 88, 12, 13, 14, 15, 16, 17, 18])
```

4.10 缺失值处理

`np.isnan()`——判断数组的每个元素是否为缺失值，这种以列为单位计算的方法叫 `ufunc`

```
In [105]: myArray = np.array([11, 12, 13, 14, 15, 16, 17, np.nan])
np.isnan(myArray)
```

```
Out[105]: array([False, False, False, False, False, False, False,  True])
```

`np.any()`——判断数组中是否至少有一个缺失值

```
In [106]: np.any(np.isnan(myArray))    # np.any 判断是否存在缺失值
```

```
Out[106]: True
```

`np.all()`——判断数组中的所有元素是否为缺失值

```
In [107]: np.all(np.isnan(myArray))    # np.all 判断是否全部都是缺失值
```

```
Out[107]: False
```

【提醒】：缺失值的处理

- 在很多函数计算中，如遇到缺失值，会报错或得到NaN值
- 处理方法是使用NaN-Safety函数，如`np.nansum()`

```
In [108]: MyArray = np.array([1, 2, 3, np.nan])
np.nansum(MyArray)    # nan-safety版本，会忽略缺失值
```

```
Out[108]: 6.0
```

【注意】：`np.nan`与`None`的却别：

- `None`是Python基础语法提供的特殊数据类型，不能参加算术运算
- `np.nan`是numpy提供的数据类型，属于float类型，可以参加算术运算

```
In [109]: np.sum(MyArray)
```

```
Out[109]: nan
```

4.11 ndarray的广播规则

当两个不同构的矩阵运算时，需要用到广播规则，

- 规则一：如果列数一样但行数不一样，那么进行以行为单位的广播操作，进行循环补齐

即：低维的向高维广播，列是低维，行是高维，如二位矩阵相加前提是低纬度（列）的维度要相同。

```
In [110]: import numpy as np
A1 = np.array(range(1, 10)).reshape([3, 3])
A1
```

```
Out[110]: array([[1, 2, 3],
                 [4, 5, 6],
                 [7, 8, 9]])
```

```
In [111]: A2 = np.array([10, 10, 10])
A2
```

```
Out[111]: array([10, 10, 10])
```

A1和A2的列数一样，行数不同。

在A1+A2操作之前，进行以行为单位的广播操作，将A1和A2转换为相同结构后进行计算

```
In [112]: A1+A2
```

```
Out[112]: array([[11, 12, 13],
                [14, 15, 16],
                [17, 18, 19]])
```

- 规则二：如果列数不一致（除了列数为1），则解释器将报错

```
In [113]: A3 = np.arange(10).reshape(2, 5)
A3
```

```
Out[113]: array([[0, 1, 2, 3, 4],
                [5, 6, 7, 8, 9]])
```

```
In [114]: A4 = np.arange(16).reshape(4, 4)
A4
```

```
Out[114]: array([[ 0,  1,  2,  3],
                [ 4,  5,  6,  7],
                [ 8,  9, 10, 11],
                [12, 13, 14, 15]])
```

```
In [115]: A3+A4 # 报错，因为两者的列数不相等
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-115-5c520ab0473e> in <module>()
----> 1 A3+A4 # 报错，因为两者的列数不相等

ValueError: operands could not be broadcast together with shapes (2,5) (4,4)
```

4.12 ndarray 的排序

```
In [116]: import numpy as np
myArray = np.array([11, 18, 13, 12, 19, 15, 14, 17, 16])
myArray
```

```
Out[116]: array([11, 18, 13, 12, 19, 15, 14, 17, 16])
```

(1) 返回排序结果——np.sort()

```
In [117]: np.sort(myArray)
```

```
Out[117]: array([11, 12, 13, 14, 15, 16, 17, 18, 19])
```

(2) 返回排序后的index——np.argsort()

```
In [118]: np.argsort(myArray) # argsort 显示排序后的元素在原列表中的下标
```

```
Out[118]: array([0, 3, 2, 6, 5, 8, 7, 1, 4], dtype=int64)
```

(3) 多维数组按指定维度排序——np.sort()中加一个轴参数axis

```
In [119]: MyArray = np.array([[21, 22, 23, 24, 25],
                             [35, 34, 33, 32, 31],
                             [1, 2, 3, 100, 4]])
MyArray
```

```
Out[119]: array([[ 21,  22,  23,  24,  25],
                 [ 35,  34,  33,  32,  31],
                 [   1,   2,   3, 100,   4]])
```

```
In [120]: np.sort(MyArray, axis = 1)
```

```
Out[120]: array([[ 21,  22,  23,  24,  25],
                 [ 31,  32,  33,  34,  35],
                 [   1,   2,   3,   4, 100]])
```

【注意】：axis = 1的含义如下：

- (1) 计算前后的行数不变
- (2) 以行为单位，每行独立计算
- (3) 逐行计算

```
In [121]: np.sort(MyArray, axis = 0)
```

```
Out[121]: array([[   1,   2,   3,  24,   4],
                 [ 21,  22,  23,  32,  25],
                 [ 35,  34,  33, 100,  31]])
```

【注意】：axis = 0 的含义如下：

- (1) 计算前后的列数不变
- (2) 以列为单位，每列独立计算
- (3) 逐列计算

5. Series

系列，Python中Series,主要用来处理数据。

Pandas包提供了一种key-value型数据结构，其中key为用户定义的显式index。

Series和Python列表的区别在于，前者的每个元素带有两种index。

- 显式index：由用户指定
- 隐式index：由系统自动分配

Series的访问既可以通过显式index，也可以通过隐式index。

5.1 主要特点

特点之一：Series式一种key-value型数据结构，每个元素由如下两部分组成。

- key：多个显式index
- value：每个显式index对应的值value

特点之二：Series有两种index

- 显式index：定义一个Series对象时，显式地指定index
- 隐式index：Series对象中每个元素的下标类似Python“列表”的下标

【补充】：

Pandas 中的Series 改进dict

dict 对集合的改进是有了显性索引

Series 对dict的改进是又加入了下标

5.2 定义方法

用pd.Series()函数，参数如下：

- data参数：对应的是Values

取值：字典、列表等

- index参数：对应的是显式index

取值：与第一个参数等长的字典、列表等

注意：若index为字符串时，需要用双引号或单引号括起来，否则会报错。

```
In [122]: import pandas as pd
mySeries1 = pd.Series([11, 12, 13, 14, 15, 16, 17], index = ["a", "b", "c", "d", "e", "f", "g"])
mySeries1
```

```
Out[122]: a    11
          b    12
          c    13
          d    14
          e    15
          f    16
          g    17
          dtype: int64
```

```
In [123]: import pandas as pd
mySeries1 = pd.Series([11, 12, 13, 14, 15, 16, 17], index = [a, b, c, d, e, f, g])
mySeries1 #【注意】报错信息为NameError: name 'a' is not defined, 原因分析：index为字符串时，忘记了用双引号或单引
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-123-75a54f20910d> in <module>()
      1 import pandas as pd
----> 2 mySeries1 = pd.Series([11, 12, 13, 14, 15, 16, 17], index = [a, b, c, d, e, f, g])
      3 mySeries1 #【注意】报错信息为NameError: name 'a' is not defined, 原因分析：index为字符串
时，忘记了用双引号或单引号括起来。

NameError: name 'a' is not defined
```

```
In [124]: mySeries2 = pd.Series([10], index = ["a", "b", "c", "d", "e", "f", "g"])
mySeries2 #【注意】：data和index的参数个数要相等
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-124-ae264d7c7b01> in <module>()
----> 1 mySeries2 = pd.Series([10], index = ["a", "b", "c", "d", "e", "f", "g"])
      2 mySeries2 #【注意】：data和index的参数个数要相等

D:\ProgramData\Anaconda3\lib\site-packages\pandas\core\series.py in __init__(self, data, index, dt
ype, name, copy, fastpath)
    290         if len(index) != len(data):
    291             raise ValueError(
--> 292                 f"Length of passed values is {len(data)}, "
    293                 f"index implies {len(index)}."
    294             )

ValueError: Length of passed values is 1, index implies 7.
```

```
In [125]: mySeries3=pd.Series([1,2,3,4,5], index=["a","b","c"])
mySeries3
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-125-a3e4ee430ecc> in <module>()
----> 1 mySeries3=pd.Series([1,2,3,4,5], index=["a","b","c"])
      2 mySeries3

D:\ProgramData\Anaconda3\lib\site-packages\pandas\core\series.py in __init__(self, data, index, dtype, name, copy, fastpath)
    290         if len(index) != len(data):
    291             raise ValueError(
--> 292                 f"Length of passed values is {len(data)}, "
    293                 f"index implies {len(index)}."
    294             )
```

ValueError: Length of passed values is 5, index implies 3.

5.3 操作方式

Series 有隐式索引的下标

(1) 查看显式index部分

```
In [126]: import pandas as pd
mySeries4=pd.Series([21,22,23,24,25,26,27], index=["a","b","c","d","e","f","g"])
mySeries4.index
```

Out[126]: Index(['a', 'b', 'c', 'd', 'e', 'f', 'g'], dtype='object')

(2) 查看values部分

```
In [127]: mySeries4.values #【注意】此处，values的拼写方法为复数
```

Out[127]: array([21, 22, 23, 24, 25, 26, 27], dtype=int64)

(3) 通过显式index查看元素，支持切片。

【注意】：key必须带引号，单引号和双引号都可以

```
In [128]: mySeries4['b']
```

Out[128]: 22

```
In [129]: mySeries4["b"]
```

Out[129]: 22

```
In [130]: mySeries4[["a","b","c"]]
# 【注意】显式index中支持切片。此处的两个方括号“[]”，分别为Series和切片的[]。
```

Out[130]: a 21
b 22
c 23
dtype: int64

```
In [131]: mySeries4["a":"d"]
# 显式index可以作为start和stop位置。
# 除了常量，显式index必须用双引号或单引号括起来。
```

Out[131]: a 21
b 22
c 23
d 24
dtype: int64

(4) 支持通过隐式index读取元素

```
In [132]: mySeries4[1:4:2]
```

```
Out[132]: b    22
          d    24
          dtype: int64
```

```
In [133]: mySeries4
```

```
Out[133]: a    21
          b    22
          c    23
          d    24
          e    25
          f    26
          g    27
          dtype: int64
```

(5) 支持显式index的in操作

```
In [134]: "c" in mySeries4 # in 操作 Series是在index中搜索
```

```
Out[134]: True
```

```
In [135]: "h" in mySeries4
```

```
Out[135]: False
```

(6) 更新显式index的方法: .reindex()

```
In [136]: mySeries4=pd.Series([21,22,23,24,25,26,27], index=["a","b","c","d","e","f","g"])
          mySeries5=mySeries4.reindex(index=["b","c","a","d","e","g","f"])
          mySeries5
          # reindex 重新索引 只是把索引的顺序改变, 但是不改变原索引与原数据的对应关系。即通过显示索引修改隐式索引
```

```
Out[136]: b    22
          c    23
          a    21
          d    24
          e    25
          g    27
          f    26
          dtype: int64
```

```
In [137]: mySeries5=mySeries4.reindex(index=["b","c","a","d","e","g","f"])
          mySeries4
```

```
Out[137]: a    21
          b    22
          c    23
          d    24
          e    25
          f    26
          g    27
          dtype: int64
```

【注意】：“更新显式index”是指调整index的次序，而不是定义一个全新的index，即更新后的index必须为已存在，否则自动增加key（显式index），对应的value为NaN缺失值。如下例：

```
In [138]: mySeries5=mySeries4.reindex(index=["new1","c","a","new2","e","g","new3"]
          )
          mySeries5
```

```
Out[138]: new1    NaN
          c      23.0
          a      21.0
          new2    NaN
          e      25.0
          g      27.0
          new3    NaN
          dtype: float64
```

【注意】：reindex()不改变Series对象本身的显式index

```
In [139]: mySeries4
```

```
Out[139]: a    21  
b    22  
c    23  
d    24  
e    25  
f    26  
g    27  
dtype: int64
```

6. DataFrame

Pandas的灵魂：DataFrame

DataFrame代表的是一种类似关系表的数据结构。

DataFrame是数据科学中最为广泛应用的数据结构之一。

数据==元组

数据框==关系表

6.1 创建方法

【补充】

pd.read和pd.read_csv返回的是数据框

数据框与数组的区别：

- 数据框有显示索引
- 数据框中可以含有非数值型数据

数据框的第一行叫：列名（columns），是列的显示索引

第一列叫：行名（index），是行的显示索引

数据框用[]切片后是Series，再切Series后是数据

数据框用[][]切片后仍是数据框

（1）创建方法一：直接定义——pd.DataFrame()（很少用）

参数可以是ndarray、列表、字典、元组、Series等

```
In [140]: import numpy as np  
import pandas as pd  
df1 = pd.DataFrame(np.arange(10).reshape(2, 5))  
df1
```

```
Out[140]:
```

	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9

（2）创建方法二：导入定义——当用Pandas包导入一个外部文件时，将自动转换为DataFrame对象

【注意】：有时导入外部文件时出现“乱码”现象，原因在于字符集编码有误，纠正方法为设置属性encoding

```
In [141]: df2 = pd.read_csv('bc_data.csv')  
df2.shape
```

```
Out[141]: (569, 32)
```

head()和tail()是数据分析和数据科学项目中常用的两个函数，分别用于显示数据框的前几行和后几行。当数据量很大时，无法也没有必要显示全部内容。

```
In [142]: df2 = df2[["id", "diagnosis", "area_mean"]]
df2.head()
```

```
Out[142]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
3	84348301	M	386.1
4	84358402	M	1297.0

6.2 查看DataFrame中的行/列

```
In [143]: #查看行的显示索引
df2.index
```

```
Out[143]: RangeIndex(start=0, stop=569, step=1)
```

```
In [144]: #计算行数
df2.index.size
```

```
Out[144]: 569
```

```
In [145]: #查看列的显式索引
df2.columns
```

```
Out[145]: Index(['id', 'diagnosis', 'area_mean'], dtype='object')
```

```
In [146]: #计算列数
df2.columns.size
```

```
Out[146]: 3
```

```
In [147]: #同时显示行数和列数，即查看DataFrame的形状
df2.shape
```

```
Out[147]: (569, 3)
```

```
In [148]: print("行数为: ", df2.shape[0])
print("列数为: ", df2.shape[1])
```

```
行数为: 569
列数为: 3
```

6.3 访问元素的方法

(1) 按列名读取

```
In [149]: #第一种写法——列名出线在下标中
df2["id"].head()
```

```
Out[149]: 0      842302
1      842517
2      84300903
3      84348301
4      84358402
Name: id, dtype: int64
```

```
In [150]: #第二种写法——可将列名当作数据框的一个属性来用
df2.id.head() # 建议采用此方式
```

```
Out[150]: 0      842302
1      842517
2      84300903
3      84348301
4      84358402
Name: id, dtype: int64
```

```
In [151]: #第三种写法——列名和行号一起用
#【注意】数据框的第0轴为列，所以不能写成df2[2]["id"]，否则报错——KeyError:2
df2["id"][2]
```

```
Out[151]: 84300903
```

```
In [152]: #第四种写法——属性名和行名一起用
df2.id[2]
```

```
Out[152]: 84300903
```

```
In [153]: #第五种写法——用切片
df2["id"][[2,4]]
```

```
Out[153]: 2      84300903
4      84358402
Name: id, dtype: int64
```

(2) 按index

.loc 表示中括号中是显示索引

.iloc 表示中括号中是隐式索引

使用loc的3个原则：

- 切片中有逗号，必须给出loc或者iloc
- 必须把行和列都给出
- 连续切片，必须行在前列在后[行][列]

```
In [154]: df2.loc[1,"id"]
```

```
Out[154]: 842517
```

(3) 按位置，即按隐式索引

```
In [155]: df2.iloc[1,0]
```

```
Out[155]: 842517
```

(4) .loc和.iloc的“老式统一写法”——.ix[]

```
In [156]: df2.ix[[1],["id"]] # .ix 先搜索显示，再搜索隐式，目前版本，该函数已经取消
```

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-156-1698alc9d275> in <module>()
----> 1 df2.ix[[1],["id"]] # .ix 先搜索显示，再搜索隐式，目前版本，该函数已经取消

D:\ProgramData\Anaconda3\lib\site-packages\pandas\core\generic.py in __getattr__(self, name)
   5272         if self._info_axis._can_hold_identifiers_and_holds_name(name):
   5273             return self[name]
-> 5274         return object.__getattr__(self, name)
   5275
   5276     def __setattr__(self, name: str, value) -> None:

AttributeError: 'DataFrame' object has no attribute 'ix'
```

按显式index访问非连续元素

```
In [157]: df2[["area_mean", "id"]].head()
```

```
Out[157]:
```

	area_mean	id
0	1001.0	842302
1	1326.0	842517
2	1203.0	84300903
3	386.1	84348301
4	1297.0	84358402

6.4 index操作

(1) DataFrame的行和列都有自己的名称，即显式index

- 行的显式index的名称: index
- 列的显式index的名称: columns

```
In [158]: df2.index
```

```
Out[158]: RangeIndex(start=0, stop=569, step=1)
```

```
In [159]: df2.columns
```

```
Out[159]: Index(['id', 'diagnosis', 'area_mean'], dtype='object')
```

(2) 可以按显式index读取

```
In [160]: df2["id"].head()
```

```
Out[160]: 0      842302
1      842517
2      84300903
3      84348301
4      84358402
Name: id, dtype: int64
```

(3) 更改显式index的方法: 用reindex()方法

【注意】: 更改显式index的含义: 并不是指重新给出新index, 而是调整index的顺序。

```
In [161]: df2.reindex(index=["1", "2", "3"], columns=["1", "2", "3"])
df2.head()
```

```
Out[161]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
3	84348301	M	386.1
4	84358402	M	1297.0

```
In [162]: df2.reindex(index=[2, 3, 1], columns=['id', 'diagnosis', 'area_mean'])
```

```
Out[162]:
```

	id	diagnosis	area_mean
2	84300903	M	1203.0
3	84348301	M	386.1
1	842517	M	1326.0

在重新索引时可以新增一个显式index。

如下例中新增的index的名称为MyNewColumn

```
In [163]: df3=df2.reindex(index=[2,3,1], columns=['id', 'diagnosis', 'area_mean', 'MyNewColumn'], fill_value = 100)
df3
```

```
Out[163]:
```

	id	diagnosis	area_mean	MyNewColumn
2	84300903	M	1203.0	100
3	84348301	M	386.1	100
1	842517	M	1326.0	100

6.5 删除或过滤行列

```
In [164]: import pandas as pd
df2 = pd.read_csv('bc_data.csv')
df2=df2[["id","diagnosis","area_mean"]]
df2.head()
```

```
Out[164]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
3	84348301	M	386.1
4	84358402	M	1297.0

```
In [165]: df2.drop([2]).head()
#此处，下标中的2为显式index，而不是隐式index
```

```
Out[165]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
3	84348301	M	386.1
4	84358402	M	1297.0
5	843786	M	477.1

【注意】：drop()方法不修改DataFrame对象本身

```
In [166]: df2.head()
```

```
Out[166]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
3	84348301	M	386.1
4	84358402	M	1297.0

```
In [167]: import pandas as pd
df2 = pd.read_csv('bc_data.csv')
df2=df2[["id","diagnosis","area_mean"]]

df2.drop([3,4],axis = 0, inplace =True) #drop默认inplace = False, 另生成值, 若改为True, 则为就地修改
df2.head()
```

Out[167]:

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
5	843786	M	477.1
6	844359	M	1040.0

【提示】：axis = 0 的含义如下：

- (1) 计算前后的列数不变
- (2) 以列为单位计算
- (3) 逐列计算

df2.drop()的第一个参数可以为列表, 也可以为一个值

```
In [168]: import pandas as pd
df2 = pd.read_csv('bc_data.csv')
df2=df2[["id","diagnosis","area_mean"]]

df2.drop([3,4],axis = 0, inplace =False) #drop默认inplace = False, 另生成值, 若改为True, 则为就地修改
df2.head()
```

Out[168]:

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
3	84348301	M	386.1
4	84358402	M	1297.0

删除列的第一种方法——用切片操作

【注】del 是Python基础语法中的语句

```
In [169]: import pandas as pd
df2 = pd.read_csv('bc_data.csv')
df2=df2[["id","diagnosis","area_mean"]]
del df2["area_mean"]
df2.head()
```

Out[169]:

	id	diagnosis
0	842302	M
1	842517	M
2	84300903	M
3	84348301	M
4	84358402	M

删除列的第二种方法——用方法drop()

【注】drop() 是Pandas提供的函数

```
In [170]: import pandas as pd
df2 = pd.read_csv('bc_data.csv')
df2=df2[["id","diagnosis","area_mean"]]
df2.drop(["id","diagnosis"], axis=1, inplace=True)
df2.head()
```

```
Out[170]:
```

	area_mean
0	1001.0
1	1326.0
2	1203.0
3	386.1
4	1297.0

按列条件过滤方法——在“下标”中写过过滤条件

```
In [171]: import pandas as pd
df2 =pd.read_csv('bc_data.csv')
df2=df2[["id","diagnosis","area_mean"]]
df2[df2.area_mean> 1000].head()
```

```
Out[171]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
4	84358402	M	1297.0
6	844359	M	1040.0

```
In [172]: df2[df2.area_mean> 1000][["id","diagnosis"]].head()
# 此处代码是过滤条件和切片的综合运用
```

```
Out[172]:
```

	id	diagnosis
0	842302	M
1	842517	M
2	84300903	M
4	84358402	M
6	844359	M

```
In [173]: df2[(df2.area_mean>1000) & (df2.diagnosis=="M")].head()
```

```
Out[173]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
4	84358402	M	1297.0
6	844359	M	1040.0

6.6 算术运算

规则之一：数据框之间的计算规则——先补齐显式index（索引）（新增索引对应值为NaN），得到相同结构后，再进行计算。

```
In [174]: df4 = pd.DataFrame(np.arange(6).reshape(2,3))
df4
```

```
Out[174]:
```

	0	1	2
0	0	1	2
1	3	4	5

```
In [175]: df5=pd.DataFrame(np.arange(10).reshape(2,5))
df5
```

```
Out[175]:
```

	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9

```
In [176]: df4 + df5 # 先补齐显示索引
```

```
Out[176]:
```

	0	1	2	3	4
0	0	2	4	NaN	NaN
1	8	10	12	NaN	NaN

规则之二：用算术运算符+、-、*等会产生NaN值，如果想将默认值填充的NaN改为指定值，建议不要使用算术运算符，而改用成员方法，如add()、sub()、mul()、div()。

```
In [177]: df6=df4.add(df5, fill_value=10)
df6
```

```
Out[177]:
```

	0	1	2	3	4
0	0	2	4	13.0	14.0
1	8	10	12	18.0	19.0

规则之三：数据框与Series的计算规则——按行（第1轴）广播，先把行改为等长，行内不做循环补齐。只是一行一行计算，不会跨行广播。

```
In [178]: s1=pd.Series(np.arange(3))
s1
```

```
Out[178]:
```

0	0
1	1
2	2

dtype: int32

```
In [179]: df6 - s1 # 带有显示索引的计算，先补齐显示索引，再进行计算
```

```
Out[179]:
```

	0	1	2	3	4
0	0.0	1.0	2.0	NaN	NaN
1	8.0	9.0	10.0	NaN	NaN

```
In [180]: df5=pd.DataFrame(np.arange(10).reshape(2,5))
s1=pd.Series(np.arange(3))
df5-s1
```

```
Out[180]:
```

	0	1	2	3	4
0	0.0	0.0	0.0	NaN	NaN
1	5.0	5.0	5.0	NaN	NaN

【提示】：等价于成员方法sub()中的axis参数取值为“1”。四则算法对应方法分别为add()、sub()、mul()、div()。

```
In [181]: df5=pd.DataFrame(np.arange(10).reshape(2,5))
print(df5,"\n")
s1=pd.Series(np.arange(3))
print(s1)
df5.sub(s1,axis=1)
```

```
   0  1  2  3  4
0  0  1  2  3  4
1  5  6  7  8  9

0    0
1    1
2    2
dtype: int32
```

Out[181]:

	0	1	2	3	4
0	0.0	0.0	0.0	NaN	NaN
1	5.0	5.0	5.0	NaN	NaN

【注意】：axis = 1 的含义可以理解为以下3个条件同时满足。

- (1) 计算前后的行数不变
- (2) 以行为单位进行计算
- (3) 逐行计算

改成按列（第0轴）计算的方法：用成员方法/函数，而不用运算符。

【注意】：计算规则：先把列做成等长，再以显示index为依据进行计算

```
In [182]: df5=pd.DataFrame(np.arange(10).reshape(2,5))
print(df5,"\n")
s1=pd.Series(np.arange(3))
print(s1)
df5.sub(s1,axis=0)
```

```
   0  1  2  3  4
0  0  1  2  3  4
1  5  6  7  8  9

0    0
1    1
2    2
dtype: int32
```

Out[182]:

	0	1	2	3	4
0	0.0	1.0	2.0	3.0	4.0
1	4.0	5.0	6.0	7.0	8.0
2	NaN	NaN	NaN	NaN	NaN

```
In [183]: df7=pd.DataFrame(np.arange(20).reshape(4,5))
df7
```

Out[183]:

	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14
3	15	16	17	18	19

In [184]: df7+2

Out[184]:

	0	1	2	3	4
0	2	3	4	5	6
1	7	8	9	10	11
2	12	13	14	15	16
3	17	18	19	20	21

In [185]: `print(df7)`
`print("df7.cumsum=",df7.cumsum())` #【注意】：按列计算时，不会进行跨列计算

	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14
3	15	16	17	18	19

df7.cumsum=

	0	1	2	3	4
0	0	1	2	3	4
1	5	7	9	11	13
2	15	18	21	24	27
3	30	34	38	42	46

In [186]: df7

Out[186]:

	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14
3	15	16	17	18	19

In [187]: # 依次计算相邻2个元素之和，即本元素与上一个元素之和。
默认情况下，按列计算，即axis = 0
`df7.rolling(2).sum()`

Out[187]:

	0	1	2	3	4
0	NaN	NaN	NaN	NaN	NaN
1	5.0	7.0	9.0	11.0	13.0
2	15.0	17.0	19.0	21.0	23.0
3	25.0	27.0	29.0	31.0	33.0

In [188]: `df7.rolling(2,axis = 1).sum()`

Out[188]:

	0	1	2	3	4
0	NaN	1.0	3.0	5.0	7.0
1	NaN	11.0	13.0	15.0	17.0
2	NaN	21.0	23.0	25.0	27.0
3	NaN	31.0	33.0	35.0	37.0

```
In [189]: df7.cov() #协方差矩阵
```

```
Out[189]:
```

	0	1	2	3	4
0	41.666667	41.666667	41.666667	41.666667	41.666667
1	41.666667	41.666667	41.666667	41.666667	41.666667
2	41.666667	41.666667	41.666667	41.666667	41.666667
3	41.666667	41.666667	41.666667	41.666667	41.666667
4	41.666667	41.666667	41.666667	41.666667	41.666667

```
In [190]: df7.corr() # 相关系数矩阵
```

```
Out[190]:
```

	0	1	2	3	4
0	1.0	1.0	1.0	1.0	1.0
1	1.0	1.0	1.0	1.0	1.0
2	1.0	1.0	1.0	1.0	1.0
3	1.0	1.0	1.0	1.0	1.0
4	1.0	1.0	1.0	1.0	1.0

```
In [191]: import pandas as pd
df2 = pd.read_csv('bc_data.csv')
df2=df2[["id","diagnosis","area_mean"]][2:5]
df2.T # 数据框的转置
```

```
Out[191]:
```

	2	3	4
id	84300903	84348301	84358402
diagnosis	M	M	M
area_mean	1203	386.1	1297

6.7 大小比较运算

```
In [192]: print(df6)
```

```
0  0  1  2  3  4
0  0  2  4 13.0 14.0
1  8 10 12 18.0 19.0
```

```
In [193]: df6>5
```

```
Out[193]:
```

	0	1	2	3	4
0	False	False	False	True	True
1	True	True	True	True	True

```
In [194]: print(s1)
```

```
0    0
1    1
2    2
dtype: int32
```

```
In [195]: df6>s1
```

```
Out[195]:
```

	0	1	2	3	4
0	False	True	True	False	False
1	True	True	True	False	False

```
In [196]: df6.T>(2,18)
```

```
Out[196]:
```

	0	1
0	False	False
1	False	False
2	True	False
3	True	False
4	True	True

6.8 统计信息

方法describe()是数据分析与数据科学项目中最常用的描述性统计方法之一。

```
In [197]: import numpy as np
import pandas as pd
df2 = pd.read_csv('bc_data.csv')
df2 = df2[["id", "diagnosis", "area_mean"]]
df2.describe()      # describe 是按列为单位计算的，只计算数值类型的列，.info()则可以进行非数值型的列
```

```
Out[197]:
```

	id	area_mean
count	5.690000e+02	569.000000
mean	3.037183e+07	654.889104
std	1.250206e+08	351.914129
min	8.670000e+03	143.500000
25%	8.692180e+05	420.300000
50%	9.060240e+05	551.100000
75%	8.813129e+06	782.700000
max	9.113205e+08	2501.000000

```
In [198]: df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   id           569 non-null    int64  
1   diagnosis    569 non-null    object  
2   area_mean    569 non-null    float64
dtypes: float64(1), int64(1), object(1)
memory usage: 13.5+ KB
```

```
In [199]: #数据框的过滤方法：将过滤条件写在“下标”中
dt = df2[df2.diagnosis=='M']
```

```
In [200]: #查看前几行数据
dt.head()
```

```
Out[200]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
3	84348301	M	386.1
4	84358402	M	1297.0

```
In [201]: #查看最后几行
dt.tail()
```

```
Out[201]:
```

	id	diagnosis	area_mean
563	926125	M	1347.0
564	926424	M	1479.0
565	926682	M	1261.0
566	926954	M	858.1
567	927241	M	1265.0

```
In [202]: #频次统计——count()方法
df2[df2.diagnosis == 'M'].count()
```

```
Out[202]: id          212
diagnosis    212
area_mean    212
dtype: int64
```

```
In [203]: #可以采用Fancy Indexing 来访问非连续的行或列
df2[["area_mean", "id"]].head()
```

```
Out[203]:
```

	area_mean	id
0	1001.0	842302
1	1326.0	842517
2	1203.0	84300903
3	386.1	84348301
4	1297.0	84358402

6.9 排序

```
In [204]: #查看数据框的当前值的前8行
df2.head(8)
```

```
Out[204]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
3	84348301	M	386.1
4	84358402	M	1297.0
5	843786	M	477.1
6	844359	M	1040.0
7	84458202	M	577.9

```
In [205]: #按值排序——sort_values()方法
df2.sort_values(by="area_mean", axis=0, ascending=True).head()
# by指按哪一列计算， axis 是方向， ascending 是按升序还是降序
```

```
Out[205]:
```

	id	diagnosis	area_mean
101	862722	B	143.5
539	921362	B	170.4
538	921092	B	178.8
568	92751	B	181.0
46	85713702	B	201.9

```
In [206]: #按显式index排序——sort_index() 方法
df2.sort_index(axis=1).head(3)  # 按索引名称排序，axis=1指按列
```

```
Out[206]:
```

	area_mean	diagnosis	id
0	1001.0	M	842302
1	1326.0	M	842517
2	1203.0	M	84300903

```
In [207]: df2.sort_index(axis = 0, ascending = False).head(3)
```

```
Out[207]:
```

	id	diagnosis	area_mean
568	92751	B	181.0
567	927241	M	1265.0
566	926954	M	858.1

6.10 导入/导出

read系列 to系列

【提示】：DataFrame导入/导出的前提——需要知道当前工作目录的位置

```
In [208]: import os
print(os.getcwd())
```

C:\Users\Admin\深度学习基础（一）2021-2022\第一次上机内容

```
In [209]: #写出方法——用系列方法.to_***(), 如.to_csv()
df2.head(3).to_csv("df2.csv")
```

```
In [210]: #读入方法——用系列方法.read_***(), 如.read_csv()
df3 = pd.read_csv('df2.csv', index_col=0)  # index_col = 0, 表示以原表的第1列作为行名；header = 0, 表示以原表的第一
```

```
In [211]: pd.read_csv?
```

```
In [212]: df3
```

```
Out[212]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0

```
In [213]: #导出方法——to_excel() 等
df2.head(3).to_excel("df3.xls")
```

```
In [214]: df3 = pd.read_excel('df3.xls', index_col=0)
df3
```

```
Out[214]:
```

	id	diagnosis	area_mean
0	842302	M	1001
1	842517	M	1326
2	84300903	M	1203

```
In [215]: pd.read_excel?
```

6.11 缺失数据处理

判断一个数据框是否为空数据框——属性.empty

```
In [216]: df3.empty
```

```
Out[216]: False
```

【注意】：Python基础语法与Pandas中对None和NaN的处理方法不同。

- 在Python基础语法中，None不能参加计算，NaN可以参加计算
- 在Pandas中，二者一样，都可以参加计算，将None自动转换为np.nan

```
In [217]: np.nan+1 #np.nan属于float类型，可以参加算术运算
```

```
Out[217]: nan
```

```
In [218]: np.nan-np.nan
```

```
Out[218]: nan
```

```
In [219]: None + 1 #【提示】报错信息为TypeError: unsupported operand type(s) for +: 'NoneType' and 'int', 原因分析: None不
#【注意】
#None是Python基础语法中的特殊数据类型，不属于数值类型，不能参加算数运算
```

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-219-8b1e12bc3630> in <module>()
----> 1 None + 1 #【提示】报错信息为TypeError: unsupported operand type(s) for +: 'NoneType' and
      2 #【注意】
      3 #None是Python基础语法中的特殊数据类型，不属于数值类型，不能参加算数运算

TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

```
In [220]: import pandas as pd
import numpy as np
A=pd.DataFrame(np.array([10,10,20,20]).reshape(2,2),columns=list("ab"),index=list("SW"))
#【注意】：list("ab")的含义为将字符串"ab"强制类型转换为列表
A
```

```
Out[220]:
```

	a	b
S	10	10
W	20	20

```
In [221]: list("ab")
```

```
Out[221]: ['a', 'b']
```

```
In [222]: B=pd.DataFrame(np.array([1,1,1,2,2,2,3,3,3]).reshape(3,3), columns=list("abc"),index=list("SWT"))
B
```

```
Out[222]:
```

	a	b	c
S	1	1	1
W	2	2	2
T	3	3	3

```
In [223]: C=A+B
C
```

```
Out[223]:
```

	a	b	c
S	11.0	11.0	NaN
T	NaN	NaN	NaN
W	22.0	22.0	NaN

【注意】：需要注意以下三个问题：

- 计算依据并非元素的对应位置，而是“以行和列的显式index为依据”计算
- 缺失值用NaN表示
- 基本计算流程：先补列索引，并补NaN，再按索引计算

```
In [224]: A.add(B, fill_value = 0)
#成员方法/函数（如add()）比运算符（如“+”）更灵活，可以设置或调整参数
#fill_value = 0 的含义：缺失值的处理方式不是补NaN，而是补0
```

```
Out[224]:
```

	a	b	c
S	11.0	11.0	1.0
T	3.0	3.0	3.0
W	22.0	22.0	2.0

```
In [225]: A.add(B, fill_value=A.stack().mean())
#fill_value=A.stack().mean()的含义为，缺失值用“均值”插补
```

```
Out[225]:
```

	a	b	c
S	11.0	11.0	16.0
T	18.0	18.0	18.0
W	22.0	22.0	17.0

```
In [226]: A.mean()
#A.mean()是按列计算单，如果想计算一个数据框（DataFrame）中全部值的均值，需要方法stack()
```

```
Out[226]: a    15.0
          b    15.0
          dtype: float64
```

```
In [227]: A.stack()
#stack()的功能为建立多级索引，是NumPy的一个常用方法
```

```
Out[227]: S a    10
          b    10
          W a    20
          b    20
          dtype: int32
```

```
In [228]: A.stack().mean()
```

```
Out[228]: 15.0
```

【注意】：DataFrame缺失值处理的四个重要函数如下：

- isnull()——判断数据框中的每个元素是否为“空”
- notnull()——判断数据框中的每个元素是否为“非空”
- dropna()——直接删除缺失值
- fillna()——设置缺失值的填补方法

```
In [229]: C
```

```
Out[229]:
```

	a	b	c
S	11.0	11.0	NaN
T	NaN	NaN	NaN
W	22.0	22.0	NaN

```
In [230]: C.isnull()
```

```
Out[230]:
```

	a	b	c
S	False	False	True
T	True	True	True
W	False	False	True

```
In [231]: C.notnull()
```

```
Out[231]:
```

	a	b	c
S	True	True	False
T	False	False	False
W	True	True	False

```
In [232]: C.dropna(axis='index')
#当axis="index"时，凡是含有缺失值的行均被删除
```

```
Out[232]:
```

	a	b	c
--	---	---	---

```
In [233]: C.fillna(0)
#此处，用0来补充所有的缺失值NaN
```

```
Out[233]:
```

	a	b	c
S	11.0	11.0	0.0
T	0.0	0.0	0.0
W	22.0	22.0	0.0

```
In [234]: C.fillna(method = "ffill")
#ffill的含义为forward fill，即向前填充
```

```
Out[234]:
```

	a	b	c
S	11.0	11.0	NaN
T	11.0	11.0	NaN
W	22.0	22.0	NaN

```
In [235]: C.fillna(method="bfill",axis=1)
#bfill的含义为backward fill，即向后填充
```

```
Out[235]:
```

	a	b	c
S	11.0	11.0	NaN
T	NaN	NaN	NaN
W	22.0	22.0	NaN

6.12 分组统计

groupby的用法如下：

(1) 后接一个圆括号和一个方括号

- 圆括号中为分组条件
- 方括号中为计算对象

(2) 再接一个方法，如sum()和mean()

即：先按()里的数据分组，再对[]中的内容按后面的方式进行统计

```
In [236]: import pandas as pd
df2 = pd.read_csv('bc_data.csv')
df2 = df2[["id", "diagnosis", "area_mean"]]
df2.head()
```

```
Out[236]:
```

	id	diagnosis	area_mean
0	842302	M	1001.0
1	842517	M	1326.0
2	84300903	M	1203.0
3	84348301	M	386.1
4	84358402	M	1297.0

```
In [237]: df2.groupby("diagnosis")["area_mean"].mean()
```

```
Out[237]: diagnosis
B      462.790196
M      978.376415
Name: area_mean, dtype: float64
```

```
In [238]: #如果想同时计算多个函数的值，那么需要用方法.aggregate()，将多个函数名以列表形式枚举在方法aggregate()的参数中
df2.groupby("diagnosis")["area_mean"].aggregate(["mean", "sum", "max", np.median])
```

```
Out[238]:
```

	mean	sum	max	median
diagnosis				
B	462.790196	165216.1	992.1	458.4
M	978.376415	207415.8	2501.0	932.0

```
In [239]: df2.groupby("diagnosis")["area_mean"].aggregate(["mean", "sum", "max", np.median]).unstack()
#此处，unstack()函数的功能为“将关系表转换为二级索引”
#stack()和unstack()是数据分析或数据科学项目中常用函数，用于建立和撤销多级索引
```

```
Out[239]:
```

	diagnosis	
mean	B	462.790196
	M	978.376415
sum	B	165216.100000
	M	207415.800000
max	B	992.100000
	M	2501.000000
median	B	458.400000
	M	932.000000

dtype: float64

【提示】：如果将第三个位置上的函数替换成自定义函数，则用方法apply()

```
In [240]: def myfunc(x):
x["area_mean"]/=x["area_mean"].sum()
return x

df2.groupby("diagnosis").apply(myfunc).head()
```

```
Out[240]:
```

	id	diagnosis	area_mean
0	842302	M	0.004826
1	842517	M	0.006393
2	84300903	M	0.005800
3	84348301	M	0.001861
4	84358402	M	0.006253

7 日期与时间

7.1 常用包与模块

日期类型：datetime等

日期类型的格式转换：dateutil、pandas等

7.2 时间和日期类型的定义方法

(1) 定义时间类型的数据

```
In [241]: import datetime as dt

myTime = dt.time(12,34,59)
print("myTime:", myTime)
print("myTime.hour:", myTime.hour)
print("myTime.minute:", myTime.minute)
print("myTime.second:", myTime.second)
```

```
myTime: 12:34:59
myTime.hour: 12
myTime.minute: 34
myTime.second: 59
```

(2) 定义日期类型的数据

【注意】：在datetime()中，year、month、day为必选，其他为可选。

```
In [242]: dt.datetime?
```

Init signature: dt.datetime(self, /, args, *kwargs)

Docstring:

datetime(year, month, day[, hour[, minute[, second[, microsecond[, tzinfo]]]])

The year, month and day arguments are required. tzinfo may be None, or an instance of a tzinfo subclass. The remaining arguments may be ints.

File: c:\users\pro7\anaconda3\lib\datetime.py

Type: type

Subclasses: datetime

```
In [243]: dt.datetime(year = 2020, month = 3, day = 3)
```

```
Out[243]: datetime.datetime(2020, 3, 3, 0, 0)
```

```
In [244]: dt.datetime(month=3, day=3, second=59)
# 【提示】报错信息如下: TypeError: Required argument 'year' (pos 1) not found, 原因分析: year为必选参数。
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-244-d21f706f7758> in <module>()
----> 1 dt.datetime(month=3, day=3, second=59)
      2 # 【提示】报错信息如下: TypeError: Required argument 'year' (pos 1) not found, 原因分析: year为必选参数。

TypeError: Required argument 'year' (pos 1) not found
```

```
In [245]: dt.datetime(year=2018, month=3, day=3, second=59)
```

```
Out[245]: datetime.datetime(2018, 3, 3, 0, 0, 59)
```

7.3 转换方法

【提示】：

在人们的日常生活中，表示日期和时间的方式有很多种，如“3th of Julv.2018”、“2019-1-3”和“2018-07-03 00:00:00”等

- 前两种属于非标准格式，不能作为dt.datetime()的参数
- 最后一种属于标准格式，可以作为dt.datetime()的参数

将非标准格式的时间数据转换为标准格式的时间数据的方法有两种

- 第一种方法：用dateutil包中的方法parser.parse()
- 第二种方法：用pandas包中的方法to_datetime()

```
In [246]: #例1
dt.datetime("3th of July, 2020")
# 【提示】报错，
# 【提示】报错信息为【TypeError: an integer is required (got type str)】，
#原因分析：“3th of July, 2020”为【非标准格式】时间数据，不能作为dt.datetime()
```

```
TypeError                                Traceback (most recent call last)
<ipython-input-246-e124ddf52c10> in <module>()
      1 #例1
----> 2 dt.datetime("3th of July, 2020")
      3 # 【提示】报错，
      4 # 【提示】报错信息为【TypeError: an integer is required (got type str)】，
      5 #原因分析：“3th of July, 2020”为【非标准格式】时间数据，不能作为dt.datetime()

TypeError: an integer is required (got type str)
```

```
In [247]: #例2
dt.datetime("2019-1-3")
## 【提示】报错信息为【TypeError: an integer is required (got type str)】，
#原因分析：“2020-1-1”为【非标准格式】时间数据，不能作为dt.datetime()的参数
```

```
TypeError                                Traceback (most recent call last)
<ipython-input-247-6219ebd4dc98> in <module>()
      1 #例2
----> 2 dt.datetime("2019-1-3")
      3 ## 【提示】报错信息为【TypeError: an integer is required (got type str)】，
      4 #原因分析：“2020-1-1”为【非标准格式】时间数据，不能作为dt.datetime()的参数

TypeError: an integer is required (got type str)
```

(1) 用dateutil包中的parser()进行日期格式的转换。

```
In [248]: from dateutil import parser
date= parser.parse("3th of July, 2018")    # parser可以将自然语言的年月日转换为计算机语言
print(date)

2018-07-03 00:00:00
```

```
In [249]: date = parser.parse("2019-1-3")
print(date)

2019-01-03 00:00:00
```

(2) 用pandas包中的to_datetime()进行格式转换。

```
In [250]: import pandas as pd
pd.to_datetime("3th of July, 2018")
```

```
Out[250]: Timestamp('2018-07-03 00:00:00')
```

```
In [251]: import pandas as pd
pd.to_datetime("2019-1-3")
```

```
Out[251]: Timestamp('2019-01-03 00:00:00')
```

7.4 显示系统的当前时间

```
In [252]: #【提示】显示系统当前时间——.datetime.now()
dt.datetime.now()
```

```
Out[252]: datetime.datetime(2021, 11, 12, 20, 26, 30, 7658)
```

```
In [253]: #【提示】显示系统当前时间——.datetime.today()
dt.datetime.today()
```

```
Out[253]: datetime.datetime(2021, 11, 12, 20, 26, 31, 148608)
```

```
In [254]: #显示周几的方法——在strftime(String FormatTime)中进行设置
now=dt.datetime.now()
now.strftime("%W"),now.strftime("%a"),now.strftime("%A"),now.strftime("%B"),now.strftime("%C"),now.strftime("%D")
```

```
Out[254]: ('45', 'Fri', 'Friday', 'November', '20', '11/12/21')
```

【注意】：参数的大小写所代表的含义不同，如%a和%A的含义不同

- %a: 本地简化星期名称
- %A: 本地完整星期名称
- %b: 本地简化的月份名称
- %B: 本地完整的月份名称
- %c: 本地相应的日期表示和时间表示
- %W: 一年中的星期数 (00~53)

7.5 计算时差

```
In [255]: #计算时差的方法——用减法
d1=dt.datetime.now()
d2=dt.datetime(year=2017,month=3,day=3)
(d1-d2).days
```

```
Out[255]: 1715
```

```
In [256]: dt.datetime?
```

7.6 时间索引

```
In [257]: import pandas as pd
index = pd.DatetimeIndex(["2018-1-1", "2019-1-2", "2018-1-3", "2018-1-4", "2018-1-5"])
data = pd.Series([1, 2, 3, 4, 5], index=index)
data
```

```
Out[257]: 2018-01-01    1
          2019-01-02    2
          2018-01-03    3
          2018-01-04    4
          2018-01-05    5
dtype: int64
```

```
In [258]: data["2018-1-2"]
```

```
Out[258]: Series([], dtype: int64)
```

```
In [259]: data["2018"] #此时已过滤掉了2019年的数据
```

```
Out[259]: 2018-01-01    1
          2018-01-03    3
          2018-01-04    4
          2018-01-05    5
dtype: int64
```

```
In [260]: data - data["2018-1-4"]
```

```
Out[260]: 2018-01-01    NaN
          2018-01-03    NaN
          2018-01-04    0.0
          2018-01-05    NaN
          2019-01-02    NaN
          dtype: float64
```

```
In [261]: data
```

```
Out[261]: 2018-01-01    1
          2019-01-02    2
          2018-01-03    3
          2018-01-04    4
          2018-01-05    5
          dtype: int64
```

```
In [262]: #在此, “freq = “D” ” 代表的是时间单位为Day (天)
          data.to_period(freq = "D")
```

```
Out[262]: 2018-01-01    1
          2019-01-02    2
          2018-01-03    3
          2018-01-04    4
          2018-01-05    5
          Freq: D, dtype: int64
```

```
In [263]: #在此, “freq = “M” ” 代表的是时间单位为Month (月)
          data.to_period(freq = "M")
```

```
Out[263]: 2018-01    1
          2019-01    2
          2018-01    3
          2018-01    4
          2018-01    5
          Freq: M, dtype: int64
```

```
In [264]: print(data[3])
          data- data[3]
```

```
4
```

```
Out[264]: 2018-01-01   -3
          2019-01-02   -2
          2018-01-03   -1
          2018-01-04    0
          2018-01-05    1
          dtype: int64
```

```
In [265]: data
```

```
Out[265]: 2018-01-01    1
          2019-01-02    2
          2018-01-03    3
          2018-01-04    4
          2018-01-05    5
          dtype: int64
```

```
In [266]: data["20180104"]
```

```
Out[266]: 2018-01-04    4
          dtype: int64
```

```
In [267]: data - data["20180104"]
```

```
Out[267]: 2018-01-01    NaN
          2018-01-03    NaN
          2018-01-04    0.0
          2018-01-05    NaN
          2019-01-02    NaN
          dtype: float64
```

7.7 period_range()函数

与Python基础语法中range()函数和NumPy中国的arange()函数类似，在Pandas中有period_range()函数，其主要参数如下：

- freq：代表的是时间单位，Y、M、D分别代表的是年、月、日
- periods：代表的是时间单位的个数
- 第一个参数为起始时间

```
In [268]: pd.period_range("2019-1", periods = 10, freq = "D")
```

```
Out[268]: PeriodIndex(['2019-01-01', '2019-01-02', '2019-01-03', '2019-01-04',
                        '2019-01-05', '2019-01-06', '2019-01-07', '2019-01-08',
                        '2019-01-09', '2019-01-10'],
                        dtype='period[D]', freq='D')
```

```
In [269]: pd.period_range("2019-1", periods = 10, freq = "M")
```

```
Out[269]: PeriodIndex(['2019-01', '2019-02', '2019-03', '2019-04', '2019-05', '2019-06',
                        '2019-07', '2019-08', '2019-09', '2019-10'],
                        dtype='period[M]', freq='M')
```

8. 可视化

```
In [270]: #常用于数据可视化的Python包(前三个常用)
#Matplotlib
#Seaborn
#pandas
#Bokeh
#Plotly
#Vispy
#Vega
#gega-lite
```

8.1 Matplotlib可视化

【提示】：Matplotlib包含两个模式

- (1) 绘图API：pyplot，在Jupyter Notebook中通常用于可视化，本实验示例采用的是matplotlib.pyplot
- (2) 集成库：pylab，是Matplotlib和SciPy、NumPy的集成库

```
In [271]: # pyplot 常用来做数据可视化
import matplotlib.pyplot as plt
%matplotlib inline
# inline为静态绘图，jupyter notebook 将matplotlib生成的图在行内显示出来，但是不能交互
#%matplotlib notebook
#notebook为交互式图，jupyter notebook 将matplotlib生成的图在行内显示出来，支持部分交互
```

```
In [272]: import pandas as pd
women = pd.read_csv('women.csv')
women.head()
```

```
Out[272]:
```

	Unnamed: 0	height	weight
0	1	58	115
1	2	59	117
2	3	60	120
3	4	61	123
4	5	62	126

【注意】：从输出结果可以看出，women中多了一个新列"Unnamed:0"

原因分析：

pd.read_csv()在读入数据时自动增加了一个index列，即参数index_col的默认值为1。

纠正方法：

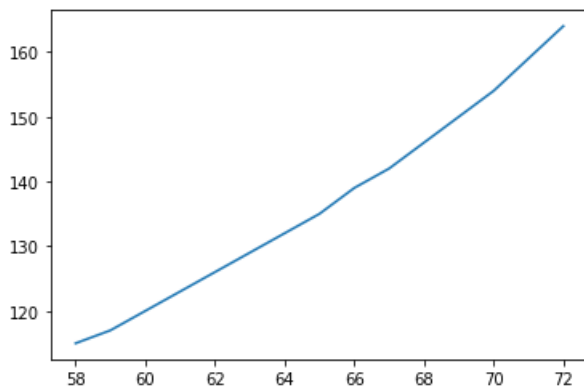
将index_col设为0。

```
In [273]: import pandas as pd
women = pd.read_csv('women.csv', index_col=0)
women.head()
```

```
Out[273]:
```

	height	weight
1	58	115
2	59	117
3	60	120
4	61	123
5	62	126

```
In [274]: #在二维坐标系上绘图: plt.plot()
plt.plot(women["height"], women["weight"])
plt.show()
```

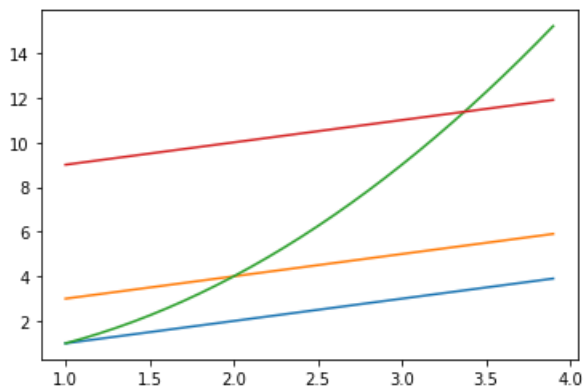


```
In [275]: #生成一个试验数据集t，用于后续操作中的可视化处理。
```

```
import numpy as np
t = np.arange(1., 4., 0.1)
t
```

```
Out[275]: array([1. , 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2. , 2.1, 2.2,
        2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9, 3. , 3.1, 3.2, 3.3, 3.4, 3.5,
        3.6, 3.7, 3.8, 3.9])
```

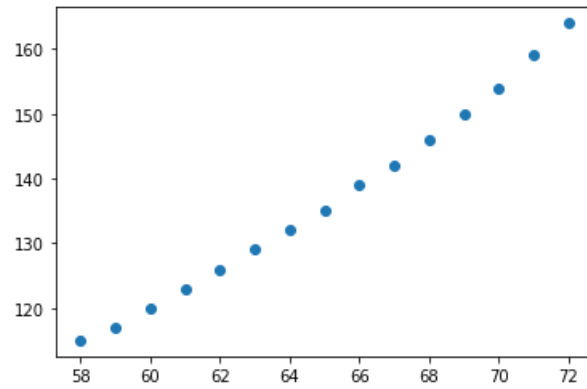
```
In [276]: #图中显示多个线条的方法为：在plt.plot()中写多个参数，参数格式为“x, y1, x, y2, x, y3, x, y4, ……”
#参数“t, t, t, t+2, t, t**2, t, t+8”的含义为“(t, t), (t, t+2), (t, t**2), (t, t+8)”，每个括号中的分别为X轴和Y轴，说明同一个X
plt.plot(t, t, t, t+2, t, t**2, t, t+8)
plt.show()
```



8.2 改变图的属性

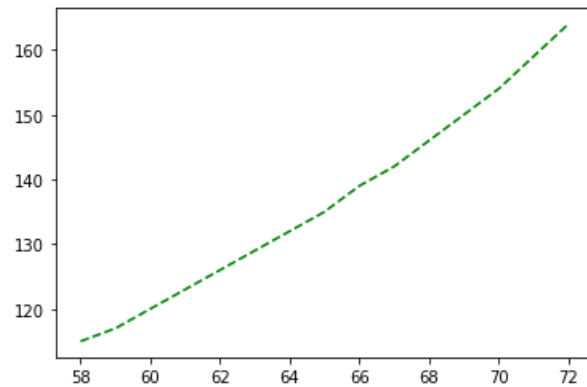
(1) 设置“点”的类型——在plt.plot()中增加第三个实参的取值，如“o”

```
In [277]: plt.plot(women["height"], women["weight"], "o")
plt.show()
```

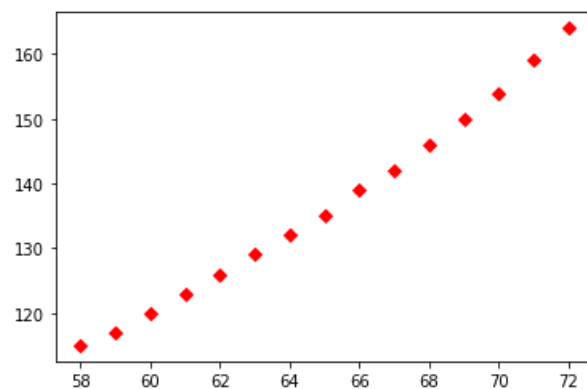


(2) 设置线的颜色与形状——修改plt.plot()的第三个实参。

```
In [278]: plt.plot(women["height"], women["weight"], "g--")
plt.show()
```



```
In [279]: plt.plot(women["height"], women["weight"], "rD")
# "rD" 的含义为 “红色 (red) + 钻石 (diamond) ”
plt.show()
```

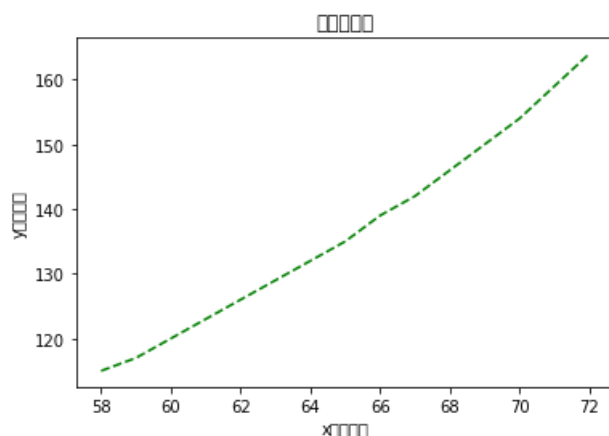


(3) 显示汉字的方法：用plt.rcParams()设置汉字字体

(4) 设置图名以及X/Y轴名称的修改，分别采用三个方法/函数：plt.title()、plt.xlabel()和plt.ylabel()

```
In [280]: plt.plot(women["height"], women["weight"], "g--")
plt.title("此处为图名")
plt.xlabel("x轴的名称")
plt.ylabel("y轴的名称")
plt.show()
```

D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:211: RuntimeWarning: Glyph 27492 missing from current font.
font.set_text(s, 0.0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:211: RuntimeWarning: Glyph 22788 missing from current font.
font.set_text(s, 0.0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:211: RuntimeWarning: Glyph 20026 missing from current font.
font.set_text(s, 0.0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:211: RuntimeWarning: Glyph 22270 missing from current font.
font.set_text(s, 0.0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:211: RuntimeWarning: Glyph 21517 missing from current font.
font.set_text(s, 0.0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:211: RuntimeWarning: Glyph 36724 missing from current font.
font.set_text(s, 0.0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:211: RuntimeWarning: Glyph 30340 missing from current font.
font.set_text(s, 0.0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:211: RuntimeWarning: Glyph 31216 missing from current font.
font.set_text(s, 0.0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:180: RuntimeWarning: Glyph 36724 missing from current font.
font.set_text(s, 0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:180: RuntimeWarning: Glyph 30340 missing from current font.
font.set_text(s, 0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:180: RuntimeWarning: Glyph 21517 missing from current font.
font.set_text(s, 0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:180: RuntimeWarning: Glyph 31216 missing from current font.
font.set_text(s, 0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:180: RuntimeWarning: Glyph 27492 missing from current font.
font.set_text(s, 0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:180: RuntimeWarning: Glyph 22788 missing from current font.
font.set_text(s, 0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:180: RuntimeWarning: Glyph 20026 missing from current font.
font.set_text(s, 0, flags=flags)
D:\ProgramData\Anaconda3\lib\site-packages\matplotlib\backends\backend_agg.py:180: RuntimeWarning: Glyph 22270 missing from current font.
font.set_text(s, 0, flags=flags)

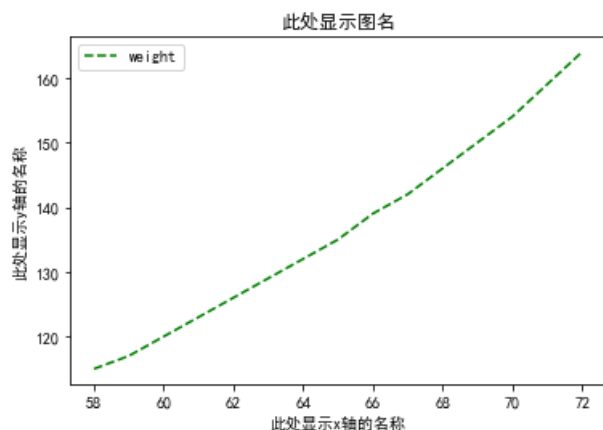


【注意】：应在调用plt.plot()之前设置字体，否则报错 【提示】：常用汉字字体有SimHei、Kaiti、Lisu、Fangsong、YouYuan等

```
In [281]: plt.rcParams['font.family'] = "SimHei"      # 可以正常显示汉字了  " SimHei "为本机中可用的字体
plt.rcParams['axes.unicode_minus']=False          # 负号可以正常显示

plt.plot(women["height"], women["weight"], "g--", label = "weight")
plt.title("此处显示图名")
plt.xlabel("此处显示x轴的名称")
plt.ylabel("此处显示y轴的名称")

plt.legend(loc = "upper left")
plt.show()
```



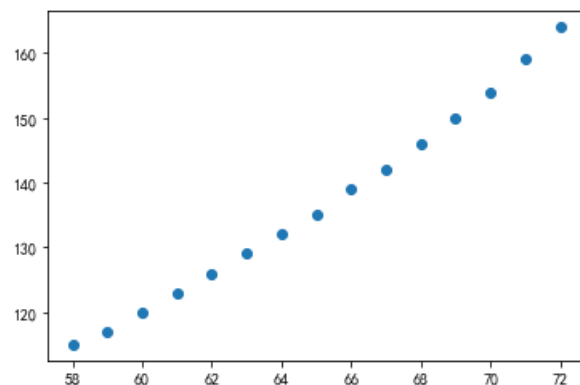
```
In [282]: plt.plot?
```

8.3 改变图的类型

将plt.plot()函数替换为其他函数，如plt.scatter()

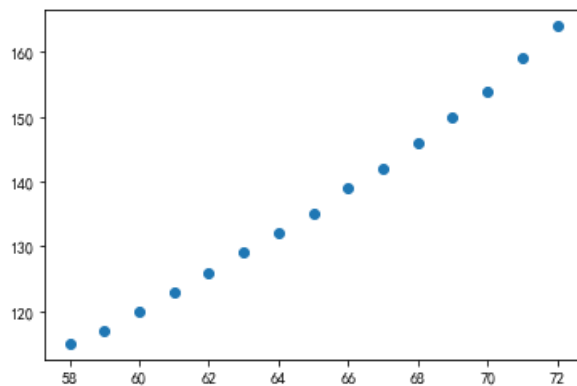
【提示】：plt.scatter()的功能为绘制散点图

```
In [283]: plt.scatter(women["height"], women["weight"])
plt.show()
```



```
In [284]: %matplotlib inline
plt.scatter(women.height, women.weight)
```

```
Out[284]: <matplotlib.collections.PathCollection at 0x2b47353b240>
```



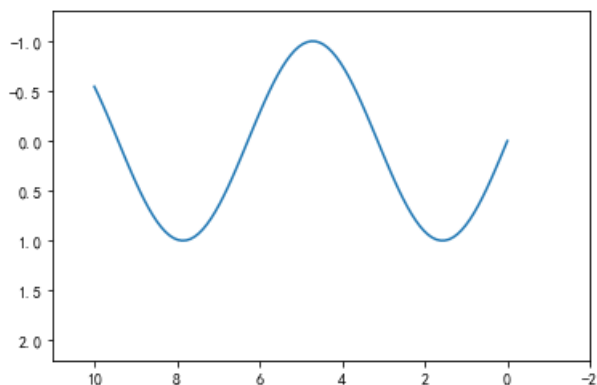
8.4 改变图的坐标轴的取值范围

横坐标和纵坐标的定义：分别采用`plt.xlim()`和`plt.ylim()`

横坐标和纵坐标同时定义：`plt.axis()`

```
In [285]: import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline
# 生成试验数据x，用于后续可视化处理
x=np.linspace(0,10,100)
# "np.linspace(0,10,100)" 的功能是返回一个含有100个元素且每个元素的取值范围为[0,10]的等距离数列
plt.plot(x,np.sin(x))
plt.xlim(11,-2) # X轴的取值范围为[11,-2]
plt.ylim(2.2,-1.3) # Y轴的取值范围为[2.2,-1.3]
```

```
Out[285]: (2.2, -1.3)
```

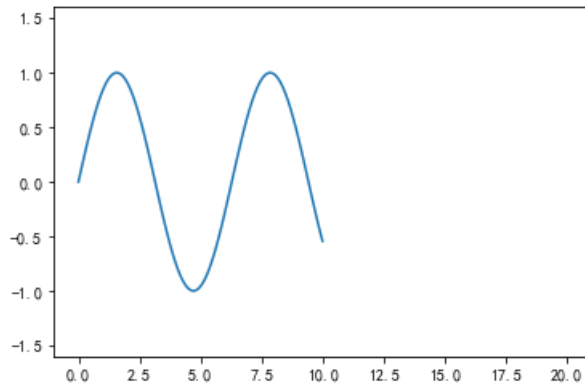


横坐标和纵坐标同时定义：`plt.axis(a1,a2,b1,b2)`。

其中，`a1`和`a2`为X轴的取值范围，`b1`和`b2`为Y轴的取值范围。

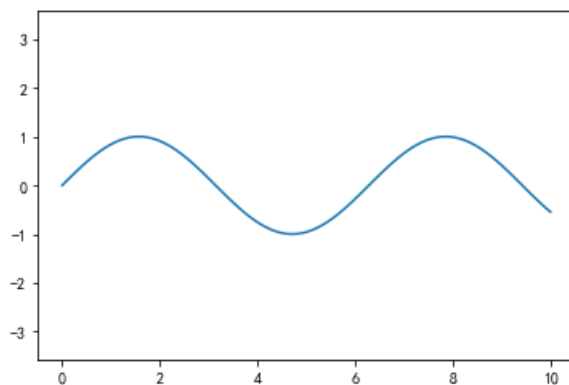
```
In [286]: plt.plot(x, np.sin(x))  
plt.axis([-1, 21, -1.6, 1.6])
```

Out[286]: [-1, 21, -1.6, 1.6]



```
In [287]: plt.plot(x, np.sin(x))  
plt.axis([-1, 21, -1.6, 1.6])  
plt.axis("equal")  
# plt.axis("equal") 的含义为X轴和Y轴的刻度单位一样
```

Out[287]: (-0.5, 10.5, -1.0993384025373631, 1.0996461858110391)

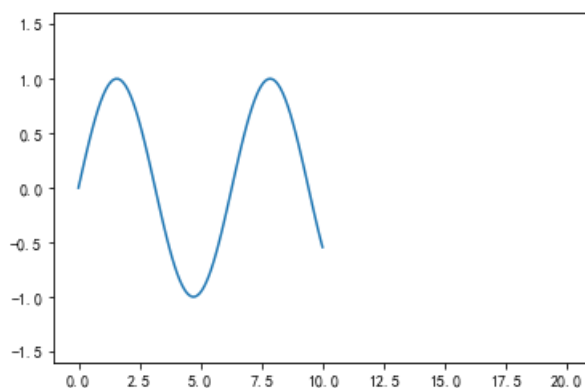


8.5 去掉边界的空白

```
plt.axis("tight")
```

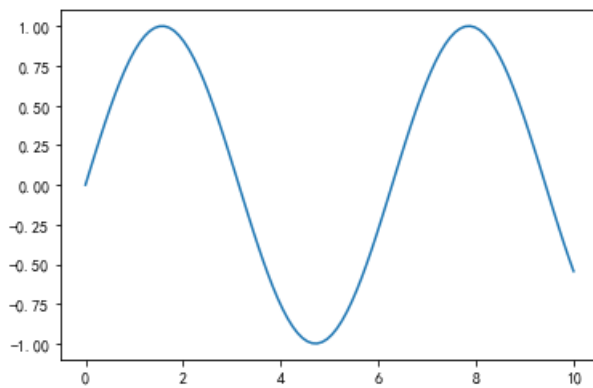
```
In [288]: plt.plot(x, np.sin(x))  
plt.axis([-1, 21, -1.6, 1.6])
```

Out[288]: [-1, 21, -1.6, 1.6]



```
In [289]: plt.plot(x, np.sin(x))
plt.axis([-1, 21, -1.6, 1.6])
plt.axis("tight") # "tight" 可以是图片显示比较紧凑
```

Out[289]: (-0.5, 10.5, -1.0993384025373631, 1.0996461858110391)



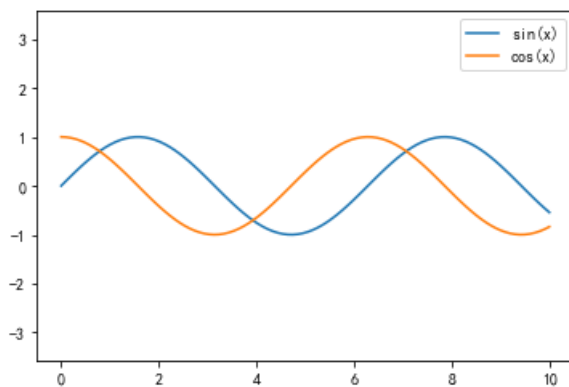
8.6 在同一个坐标上画两个图

【提示】

分别定义label (标签) , plt.legend()显示多个labels。

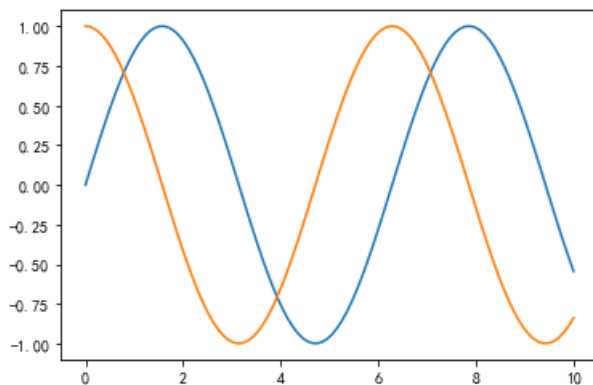
```
In [290]: plt.plot(x, np.sin(x), label = "sin(x)")
plt.plot(x, np.cos(x), label = "cos(x)")
plt.axis("equal")
plt.legend() #如果没有此行, labels名称将无法显示
```

Out[290]: <matplotlib.legend.Legend at 0x2b4732a8c18>



```
In [291]: plt.plot(x, np.sin(x), label="sin(x)")
plt.plot(x, np.cos(x), label="cos(x)")
```

Out[291]: [<matplotlib.lines.Line2D at 0x2b473412710>]



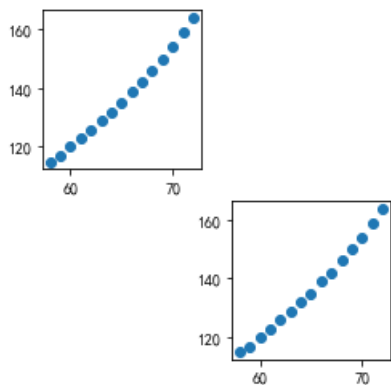
8.7 多图显示

【提示】

每个plot之前加一个subplot(x,y,z)

- x: 行数
- y: 列数
- z: 接下来的plot()的画图位置，即放在第几个“子窗口”中

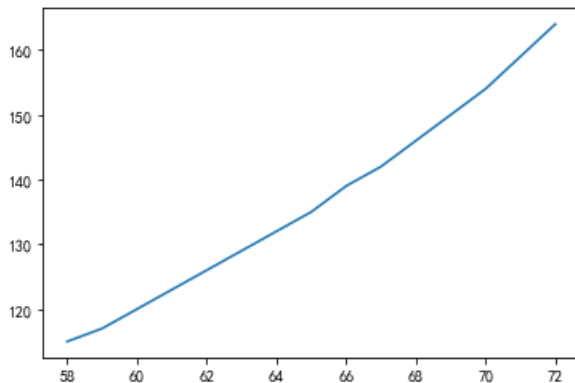
```
In [292]: plt.subplot(2,3,5)
# plt.subplot(2,3,5)的含义为“接下来的显示位置是2*3个窗口的第5个子窗口”。
plt.scatter(women.height, women.weight)
plt.subplot(2,3,1)
# plt.subplot(2,3,1)的含义为“接下来的显示位置是2*3个窗口的第1个子窗口”。从1开始编号
plt.scatter(women.height, women.weight)
plt.show()
```



8.8 图的保存

用Matplotlib提供的函数，如plt.savefig()

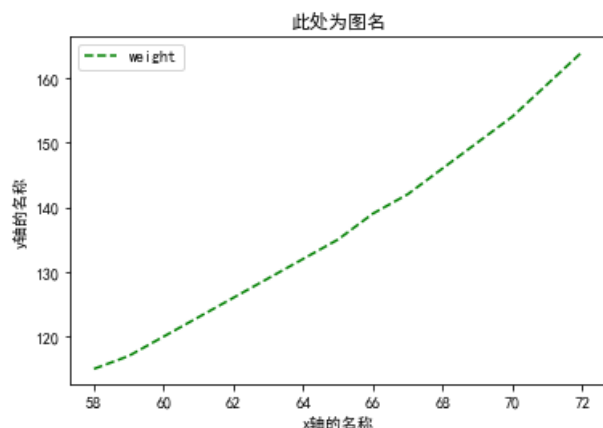
```
In [293]: women = pd.read_csv('women.csv')
plt.plot(women.height, women.weight)
plt.savefig("sagefig.png") #在此，以“sagefig.png”为文件名，保存在“当前工作目录”中
```



```
In [294]: plt.rcParams['font.family'] = "SimHei" # 可以正常显示汉字了 "SimHei" 为本机中可用的字体
plt.rcParams['axes.unicode_minus']=False # 负号可以正常显示

plt.plot(women["height"], women["weight"], "g--", label = "weight")
plt.title("此处为图名")
plt.xlabel("x轴的名称")
plt.ylabel("y轴的名称")

plt.legend(loc = "upper left")
plt.savefig("sagefig1.png")
```



8.9 散点图的画法

【提示】

首先，生成即将用于可视化的试验数据集X和Y。

在本例中使用的生成方法：Python的sklearn.datasets.samples_generator模块提供的函数make_blobs()

(1) make_blobs：生成符合正态分布的随机数据集

- 参数：

n_samples:样本数量，即行数

n_features:每个样本的特征数量，即列数

centers：类别数

random_state:随机数的生成方式

cluster_std:每个类别的方差

- 返回值，有两个：

X：测试集，类型为数组，形状为[n_samples,n_features]

y：每个成员的标签（label），也是个数组，形状为[n_samples]的数组

(2) plt.scatter()函数的参数如下：

X[:,0]和X[:,1]分别为x坐标和y坐标，分别是读取数据框X的第0列和第1列

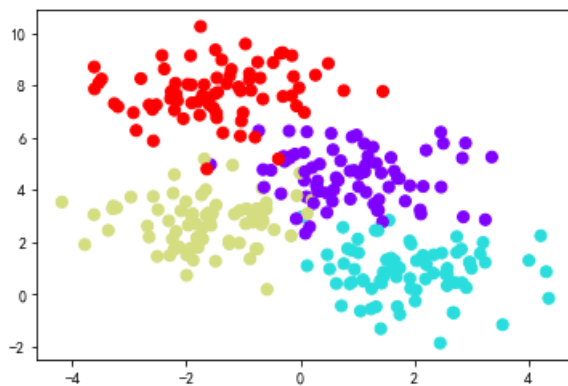
c为颜色

s为点的大小

cmap为色带，是c的补充

```
In [295]: from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples = 300, centers = 4, random_state = 0, cluster_std = 1.0)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap = "rainbow")
```

Out[295]: <matplotlib.collections.PathCollection at 0x2b473f7c7f0>

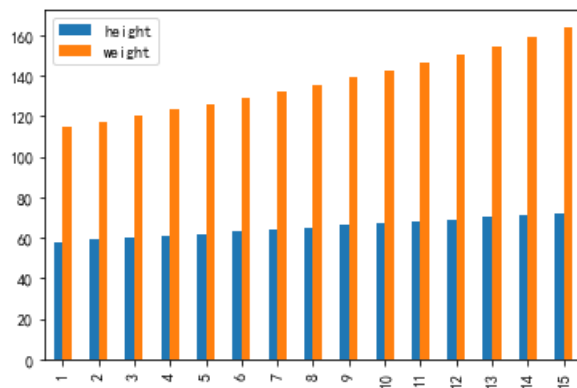


8.10 Pandas 可视化

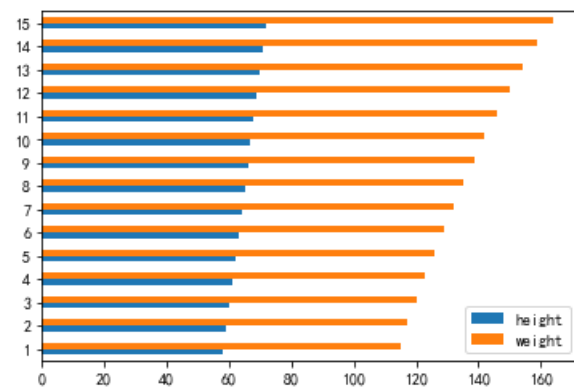
在数据分析与数据科学项目中，还可以直接用Pandas的画图函数，它继承和优化了 Matplotlib，是DataFrame（数据框）类数据的可视化处理更容易。

Pandas中可以对数据框进行可视化，具体方法：数据框名.plot()。该方法的参数 kind 决定图的类型。

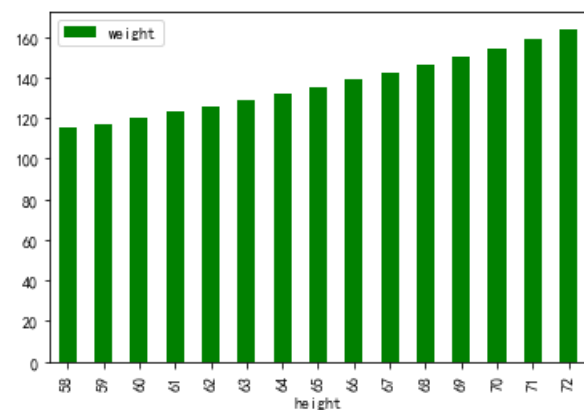
```
In [296]: import pandas as pd
women = pd.read_csv('women.csv', index_col = 0)
women.plot(kind="bar")
plt.show()
```



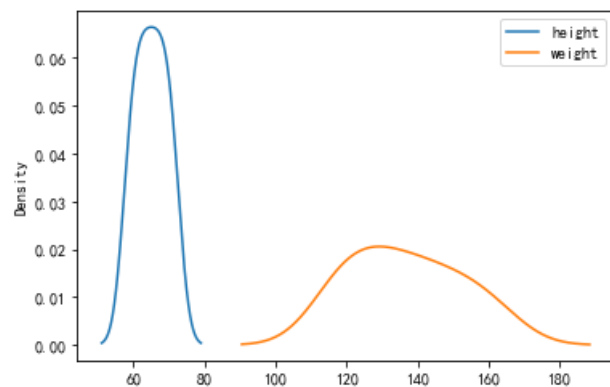
```
In [297]: women.plot(kind = "barh") # barh代表的是横向柱状图
plt.show()
```



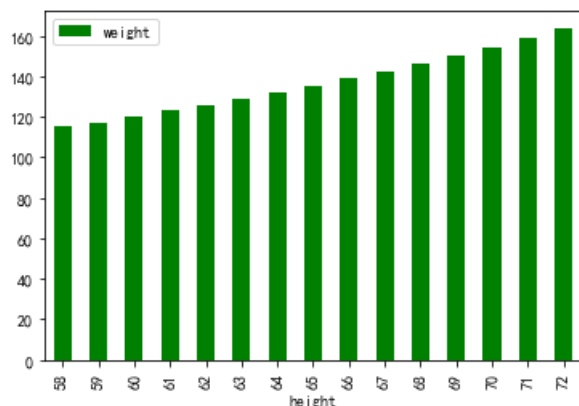
```
In [298]: women.plot(kind="bar", x="height", y="weight", color="g")
plt.show()
```



```
In [299]: women.plot(kind="kde") # ked代表的是核密度估计曲线
plt.show()
```



```
In [300]: women.plot(kind="bar", x="height", y="weight", color="g")
plt.legend(loc="best") # 图例位置为“最优”
plt.show()
```



8.11 Seaborn可视化

Seaborn可以实现比Matplotlib更美观的可视化效果，但需要与Matplotlib结合使用。

```
In [301]: import matplotlib.pyplot as plt
plt.style.use("classic") #设置背景样式
%matplotlib inline
import numpy as np
import pandas as pd
```

```
In [302]: #用plt.style.available查出可用样式列表如下
plt.style.available
```

```
Out[302]: ['bmh',
'classic',
'dark_background',
'fast',
'fivethirtyeight',
'ggplot',
'grayscale',
'seaborn-bright',
'seaborn-colorblind',
'seaborn-dark-palette',
'seaborn-dark',
'seaborn-darkgrid',
'seaborn-deep',
'seaborn-muted',
'seaborn-notebook',
'seaborn-paper',
'seaborn-pastel',
'seaborn-poster',
'seaborn-talk',
'seaborn-ticks',
'seaborn-white',
'seaborn-whitegrid',
'seaborn',
'Solarize_Light2',
'tableau-colorblind10',
'_classic_test']
```

```
In [303]: # 数据准备
rng = np.random.RandomState(0)
x=np.linspace(0, 10, 500) #生成500个0~10之间的数
y=np.cumsum(rng.randn(500, 6), 0)
```

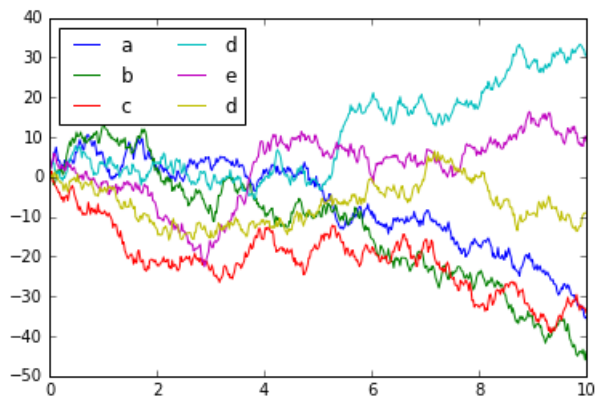
【提示】

.cumsum是Matlab中的一个函数，通常用于计算一个数组各行的累加值，语法为：

B = cumsum(A,dim), 或 B = cumsum(A)

```
In [304]: plt.plot(x, y)
plt.legend("abcde", ncol=2, loc="upper left")
```

Out[304]: <matplotlib.legend.Legend at 0x2b47354c908>



```
In [305]: #用Seaborn更美观
import seaborn as sns
sns.set()
plt.plot(x, y)
plt.legend("abcdef", ncol=2, loc="upper left")
```

Out[305]: <matplotlib.legend.Legend at 0x2b47430c320>

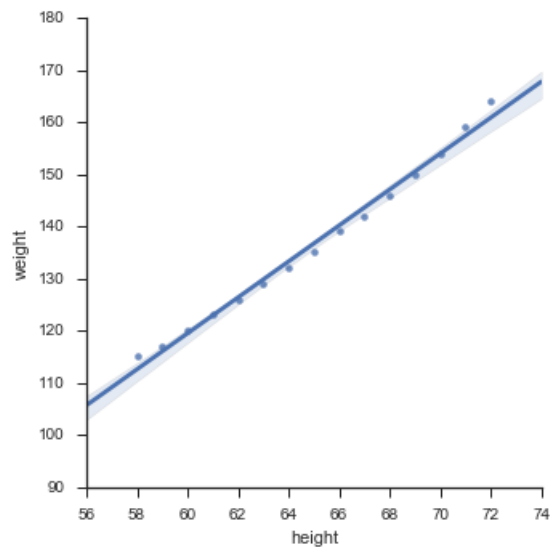


【提示】：

plt.legend()的功能为设置图例参数，分别为图例内容 ("abcdef")、图例列数 (ncol = 2) 和图例的显示位置 (loc = "upper left")。

```
In [306]: # 仅使用seaborn画图
import pandas as pd
import seaborn as sns
sns.set(style = "ticks")
df_women = pd.read_csv('women.csv', index_col = 0, header = 0)
sns.lmplot(x = "height", y = "weight", data = df_women)
```

Out[306]: <seaborn.axisgrid.FacetGrid at 0x2b474129a90>



8.12 数据可视化实战

(1)数据准备

```
In [307]: import os
os.getcwd()
```

Out[307]: 'C:\\Users\\Admin\\深度学习基础（一）2021-2022\\第一次上机内容'

```
In [308]: import pandas as pd
salaries = pd.read_csv('salaries.csv', index_col = 0)
# index_col = 0的含义为，在准备读入的数据文件salaries.csv中带有索引列且索引列位于第0列。
salaries.head(6)
```

Out[308]:

	rank	discipline	yrs.since.phd	yrs.service	sex	salary
1	Prof	B	19	18	Male	139750
2	Prof	B	20	16	Male	173200
3	AsstProf	B	4	3	Male	79750
4	Prof	B	45	39	Male	115000
5	Prof	B	40	41	Male	141500
6	AssocProf	B	6	6	Male	97000

(2) 导入Python包

```
In [309]: import seaborn as sns
```

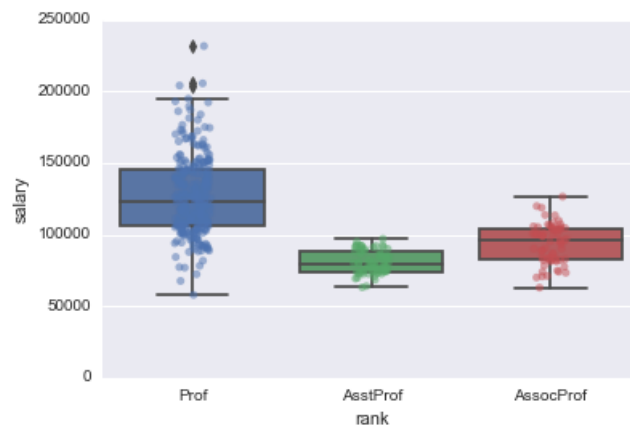
(3) 可视化绘图

```
In [310]: sns.set_style('darkgrid') #设置样式或主题为灰色+网格
```

```
In [311]: sns.stripplot(data = salaries, x = 'rank', y = 'salary', jitter = True, alpha = 0.5)
#绘制散点图，其中jitter 控制三点是否有抖动，alpha 为透明度。

sns.boxplot(data = salaries, x = 'rank', y = 'salary') #绘制箱线图

plt.savefig('salaries.png')
```



参考文献:

1. 朝乐门. Python编程：从数据分析到数据科学. 电子工业出版社，2019.